

The Python Problem-Solving Mastery Guide

A complete, self-contained guide to going from "I dread LeetCode" to passing any CoderPad / LeetCode / HackerRank style interview — with data structure fundamentals, algorithm patterns, AI/ML/LLM context, OOP, system design, and ~180 curated problems (hints in-place, all solutions at the end).

TABLE OF CONTENTS

Part I — Mindset & How to Use This Guide

Part II — Data Structures: What They Are, Why They Exist, and Where They Show Up (including in AI/ML, LLMs, RAG, ANNs, CNNs, RNNs) - Arrays & Lists - Strings - Hashmaps & Sets - Stacks & Queues - Linked Lists - Trees & Binary Search Trees - Heaps / Priority Queues - Graphs - Tries - Union-Find (Disjoint Set)

Part III — Algorithm Patterns You Must Know - Two Pointers - Sliding Window - Binary Search - Recursion & Backtracking - Dynamic Programming - Greedy - BFS & DFS - Sorting Fundamentals - Bit Manipulation - Divide & Conquer

Part IV — Python-Specific Power Tools for Interviews

Part V — Object-Oriented Programming & Design Patterns

Part VI — System Design Primer

Part VII — Interview Meta-Skills (how to think out loud, complexity, tradeoffs)

Part VIII — The Problem Set (~180 problems with hints, no solutions)

Part IX — Solutions (all problems solved with explanations)

PART I — MINDSET & HOW TO USE THIS GUIDE

The truth nobody tells you

Six years (or ten, or twenty) of building real software does not automatically make you good at LeetCode-style problems. That's not a bug in you — it's a fact about the format. Interview problems test a *narrow, specific* skill: pattern recognition under time pressure, on toy problems, without the context you normally use to think. It's a skill, and skills are trainable.

If you hate this kind of problem, it's almost always for one of three reasons:

- 1. **The problems feel disconnected from real work.** (They are — but the *underlying patterns* aren't.)
- 2. **You've been dropped into medium/hard problems without foundation.** (Motivation dies when every problem feels impossible.)
- 3. **You practice by suffering.** (Sitting stuck for 45 minutes trains dread, not skill.)

This guide fixes all three.

How to actually use this guide

Rule 1: Read the concept before touching the problems. Every data structure and pattern section in Parts II–III explains *why* it exists, *when* it's the right choice, and — importantly — *where it shows up in modern software, including AI/ML systems*. This context is the difference between memorizing and understanding.

Rule 2: Time-box every problem to 20 minutes before hint, 40 minutes before solution. Not because "you should be faster" — because sitting frustrated past that point trains avoidance. Struggle is fine; suffering is not.

Rule 3: 2-3 problems per session, max, at first. Motivation is finite. Finishing 3 problems feeling capable beats attempting 8 feeling wrecked.

Rule 4: After every problem, name the pattern. Say it out loud: "*That was a hashmap-for-lookup problem.*" You're not memorizing 180 answers — you're building an index of ~15 patterns that cover 90% of interview questions.

Rule 5: Re-solve old problems from memory. Every Friday, redo one problem you solved on Monday. Retrieval, not review, is what makes knowledge stick.

Rule 6: Never look at the solution before writing *some* code. Even bad code. Even wrong code. Writing bad code and fixing it is how you learn; reading correct code and nodding is how you forget.

Suggested pacing

Week	Focus
Week 1	Read Part II (data structures) + do 10 Easy problems
Week 2	Read Part III (patterns) + do 15 Easy problems
Week 3	Do 15 Easy-Medium problems, revisit any old ones you failed
Week 4–6	15 Medium problems per week, mix of categories
Week 7–8	Hard problems + system design section + mock interviews

At ~4–5 hours per week, you'll finish this document in 2 months and be genuinely interview-ready.

PART II — DATA STRUCTURES: THE FOUNDATIONS

Every problem you'll ever solve reduces to picking the right container for your data. Get this right and half the problem is solved before you write any logic.

2.1 Arrays & Lists

What it is

A contiguous block of memory holding elements you access by index. In Python, `list` is a dynamic array (auto-resizes), which combines simplicity with the "just works" ergonomics you use every day.

Complexity cheatsheet

Operation	Time
Access by index <code>arr[i]</code>	$O(1)$
Append <code>arr.append(x)</code>	$O(1)$ amortized
Insert at position <code>arr.insert(0, x)</code>	$O(n)$
Search <code>x in arr</code>	$O(n)$
Delete by value <code>arr.remove(x)</code>	$O(n)$
Delete by index <code>arr.pop(i)</code>	$O(n)$ ($O(1)$ if last)

When to reach for it

Any time you need ordered, index-addressable data. Default choice unless you have a specific reason otherwise.

Where it shows up in AI/ML

- **Every tensor is an array under the hood.** A NumPy array, PyTorch tensor, or TensorFlow tensor is a multi-dimensional array with additional metadata (shape, dtype, device). When you feed a batch of images to a CNN, that batch is a 4D array: `[batch_size, channels, height, width]`.
- **Embeddings** — the numerical representations of tokens/words/documents in LLMs and RAG — are just fixed-length arrays of floats (typically 384, 768, 1536, or 3072 dimensions). Similarity between two embeddings is dot-product or cosine similarity over these arrays.
- **Model weights** are stored as arrays. A single transformer layer contains dozens of weight matrices, each a 2D array with millions of parameters.

Python idioms worth memorizing

```
# Slicing (view, but list slicing creates copy)
arr[start:stop:step]
arr[::-1]           # reverse
arr[::2]           # every other element

# Enumeration
for i, val in enumerate(arr):
    ...

# List comprehension (faster than for loop + append)
squares = [x*x for x in arr]
evens   = [x for x in arr if x % 2 == 0]

# Sorting
arr.sort()           # in-place
sorted_arr = sorted(arr) # returns new
arr.sort(key=lambda x: -x) # custom key, descending
```

2.2 Strings

What it is

An immutable sequence of characters. Immutability matters: `s += "x"` creates a whole new string every time ($O(n)$), which is why building strings in a loop is a classic beginner trap. Use `"".join(list)` instead.

Complexity cheatsheet

Operation	Time
Access <code>s[i]</code>	$O(1)$
Concatenation <code>s + t</code>	$O(n + m)$
Substring search <code>t</code> in <code>s</code>	$O(n \cdot m)$ worst, usually near-linear
<code>s.split()</code>	$O(n)$
<code>"".join(list)</code>	$O(\text{total length})$

When to reach for it

Whenever the data is text. Most string problems are really disguised array problems — treat a string as an array of characters and you're already halfway there.

Where it shows up in AI/ML

- **LLMs eat and produce strings, but internally work on tokens.** Tokenization converts a string like `"hello world"` into a list of integer token IDs like `[15496, 995]` via algorithms like Byte-Pair Encoding (BPE) or SentencePiece. Every prompt you send to ChatGPT or Claude goes through this string-to-integer-array conversion first.
- **RAG systems** chunk long documents into overlapping string segments (typically 200–1000 tokens each) before embedding them.
- **Prompt engineering** is fundamentally string manipulation — templating, formatting, injection defense, escape handling.

Python idioms

```
# The .join() trick - 10-100x faster than += in a loop
result = "".join(parts)

# Efficient character counting
from collections import Counter
counts = Counter(s)

# Common transforms
s.lower(), s.upper(), s.strip(), s.replace("a", "b")
s.split(",") # split on delimiter
s.startswith("http"), s.endswith(".py")

# f-strings (best formatting in Python)
f"Name: {name}, Score: {score:.2f}"
```

2.3 Hashmaps (dict) & Sets

What it is

A hash table: keys are hashed to bucket locations for near-instant lookup. In Python, `dict` is a hashmap and `set` is a hashmap without values (just membership).

Complexity cheatsheet

Operation	Time
Lookup <code>d[k]</code> or <code>k in d</code>	$O(1)$ average, $O(n)$ worst
Insert <code>d[k] = v</code>	$O(1)$ average
Delete <code>del d[k]</code>	$O(1)$ average

When to reach for it

The moment you find yourself asking any of: - "Have I seen this before?" → `set` - "How many times has X appeared?" → `Counter` - "What value is associated with this key?" → `dict` - "Can I look up X in $O(1)$ instead of scanning?" → `dict/set`

This is the single most valuable pattern in interviews. If you're doing a nested loop to find pairs, you probably want a hashmap.

Where it shows up in AI/ML

- **Vector databases** (Pinecone, Weaviate, Chroma, FAISS) that power RAG use hashmap-like index structures — often combined with tree/graph structures — to look up nearest-neighbor embeddings without scanning millions of vectors linearly.
- **Vocabulary lookup in LLMs** — the mapping from token string → token ID (and back) is a hashmap. GPT-4's tokenizer holds ~100k entries in a hashmap.
- **Feature engineering** in classical ML frequently uses `dict` for one-hot encoding, target encoding, and mapping categorical values to integers.
- **Caching** (memoization in dynamic programming, KV-cache in transformer inference) is all hashmap-driven.

Python idioms

```
from collections import Counter, defaultdict

# Counter - frequency counting
counts = Counter("hello") # {'h':1, 'e':1, 'l':2, 'o':1}
counts.most_common(2)    # top-2

# defaultdict - no more KeyError checks
groups = defaultdict(list)
for word in words:
    groups[len(word)].append(word)

# dict comprehension
{k: v*2 for k, v in d.items()}

# Set operations
a & b    # intersection
a | b    # union
a - b    # difference
a ^ b    # symmetric difference
```

2.4 Stacks & Queues

What it is

- **Stack**: Last-In-First-Out (LIFO). Think of a stack of plates.
- **Queue**: First-In-First-Out (FIFO). Think of a line at the grocery store.

In Python: - Stack: just use a `list` with `append()` (push) and `pop()` (pop). - Queue: use `collections.deque` with `append()` and `popleft()` — a normal list's `pop(0)` is $O(n)$!

Complexity

Both give $O(1)$ push and pop when used correctly.

When to reach for it

- **Stack**: any "most recent unmatched" problem — balanced parentheses, undo/redo, DFS with iteration, expression evaluation, monotonic stack problems (next greater element).
- **Queue**: BFS in graphs/trees, level-order traversal, sliding window with dequeue, scheduling.

Where it shows up in AI/ML

- **The call stack itself** is how every recursive neural network operation (backprop) unwinds — automatic differentiation frameworks build a computation graph and traverse it in reverse using stack-like structures.
- **BFS/DFS in graph neural networks (GNNs)** for message passing uses queues and stacks.
- **Beam search** in LLM decoding maintains a priority queue of the top-k most likely partial sequences at each step.

Python idioms

```

from collections import deque

# Stack (LIFO)
stack = []
stack.append(1)      # push
stack.pop()          # pop last

# Queue (FIFO)
queue = deque()
queue.append(1)       # enqueue
queue.popleft()       # dequeue

# Double-ended queue
queue.appendleft(0)   # push to front
queue.pop()           # pop from back

```

2.5 Linked Lists

What it is

A chain of nodes where each node holds a value and a pointer to the next node. Unlike arrays, elements are *not* contiguous in memory.

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

```

Complexity cheatsheet

Operation	Time
Access by index	O(n)
Insert at head	O(1)
Insert at tail (if we have tail ptr)	O(1)
Delete a node (given the node)	O(1)
Search	O(n)

When to reach for it

Honestly? In real work, rarely — arrays are almost always better in practice due to cache locality. But **interviews love linked lists** because they test pointer manipulation cleanly. Master the classic patterns (reverse, cycle detection, merge, find middle) and you're set.

Where it shows up in AI/ML

- **Computation graphs** (the graphs frameworks like PyTorch build during forward passes for autograd) are essentially linked structures of operations.
- **Attention mechanisms** don't use linked lists directly, but they solve the fundamental *sequential dependency* problem that RNNs (which are conceptually linked-list-like) had.
- Historically, **RNNs and LSTMs** processed sequences token-by-token in a linked-list-style traversal — this sequential bottleneck is exactly why transformers (which process all tokens in parallel) won.

Must-know patterns

- **Two-pointer (slow/fast)**: cycle detection, finding middle, finding kth from end
- **Reversal**: iterative and recursive
- **Merge**: merging two sorted lists
- **Dummy head**: simplifies edge cases when the head might change

2.6 Trees & Binary Search Trees (BSTs)

What it is

A hierarchical structure of nodes with a root, where each node has 0+ children. A **binary tree** restricts children to at most 2 (left, right). A **BST** additionally maintains: `left.val < node.val < right.val` for every node.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Complexity (for balanced BST)

Operation	Time
Search	O(log n)
Insert	O(log n)
Delete	O(log n)
Traversal (any)	O(n)

Worst case for unbalanced BST is O(n) — this is why real databases use *self-balancing* variants (AVL, red-black, B-trees).

When to reach for it

- Hierarchical data (file systems, org charts, DOM, syntax trees)
- Ordered data where you need efficient search + insert (BST)
- Recursive problem structure (any problem where you divide the input in half repeatedly)

Where it shows up in AI/ML

- **Decision trees** (and their ensemble children — Random Forest, XGBoost, LightGBM) are literally trees. XGBoost was the dominant winning algorithm on Kaggle for years and still is on most tabular data.
- **Merkle trees** are how many distributed ML systems verify model integrity and dataset provenance.
- **Beam search trees** in LLM decoding explore possible next-token sequences as a tree, pruning low-probability branches.
- **Monte Carlo Tree Search (MCTS)** is the core algorithm behind AlphaGo, AlphaZero, and modern reasoning systems that "think" by exploring a tree of possibilities.
- **Abstract syntax trees (ASTs)** are used to parse code — critical for code-generation LLMs that need to produce syntactically valid output.

Traversals to memorize cold

```
# Depth-First Search (DFS)
def inorder(node): # Left, Root, Right – gives sorted for BST
    if not node: return
    inorder(node.left)
    print(node.val)
    inorder(node.right)

def preorder(node): # Root, Left, Right – good for copying a tree
    if not node: return
    print(node.val)
    preorder(node.left)
    preorder(node.right)

def postorder(node): # Left, Right, Root – good for deleting a tree
    if not node: return
    postorder(node.left)
    postorder(node.right)
    print(node.val)

# Breadth-First Search (BFS) – level by level
from collections import deque
def bfs(root):
    if not root: return
    q = deque([root])
    while q:
        node = q.popleft()
        print(node.val)
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
```

2.7 Heaps / Priority Queues

What it is

A specialized tree-based structure (usually implemented as an array) that maintains one invariant: the smallest (min-heap) or largest (max-heap) element is always at the root. Python's `heapq` module is a min-heap.

Complexity

Operation	Time
Push	$O(\log n)$
Pop min/max	$O(\log n)$
Peek min/max	$O(1)$
Heapify a list	$O(n)$

When to reach for it

The tell: **"top K" or "smallest/largest at each step"**. Examples: - Top K frequent elements - Median of a stream - Merge K sorted lists - Dijkstra's algorithm (shortest path) - Task scheduling by priority

Where it shows up in AI/ML

- **Beam search** in LLMs uses a heap to maintain the top-K candidate sequences during decoding — this is how models like GPT trade off quality vs. diversity.
- **Top-K sampling** during LLM generation (e.g., "sample from the 40 most likely next tokens") uses heaps.
- **Approximate nearest neighbor (ANN)** search for vector databases uses heap-backed algorithms like HNSW.
- **A* search** and other pathfinding algorithms used in RL and robotics rely on priority queues.

Python idioms

```
import heapq

heap = []
heapq.heappush(heap, 3)
heapq.heappush(heap, 1)
heapq.heappush(heap, 2)
smallest = heapq.heappop(heap)  # 1

# Get n smallest / largest
heapq.nsmallest(3, [5,1,8,2,9])  # [1, 2, 5]
heapq.nlargest(3, [5,1,8,2,9])   # [9, 8, 5]

# Max-heap: negate values
heapq.heappush(max_heap, -val)
```

2.8 Graphs

What it is

A set of vertices (nodes) connected by edges. Edges can be directed or undirected, weighted or unweighted.

Representations: - **Adjacency list** (default, most efficient for sparse graphs): `{node: [neighbors]}` - **Adjacency matrix** (fast edge lookup, uses $O(V^2)$ space): 2D array - **Edge list**: `[(u, v, weight), ...]`

Complexity (adjacency list, V vertices, E edges)

Algorithm	Time
BFS	$O(V + E)$
DFS	$O(V + E)$
Dijkstra (with heap)	$O((V + E) \log V)$
Topological sort	$O(V + E)$
Union-Find operations	$\sim O(\alpha(n))$ — practically $O(1)$

When to reach for it

Any relationship-based data: social networks, road maps, dependency chains, prerequisite courses, web pages linking to each other, molecules, workflows.

Where it shows up in AI/ML — this is huge

- **Graph Neural Networks (GNNs)** operate directly on graph structures. Used in drug discovery, protein folding (AlphaFold), fraud detection, and recommendation systems.
- **The transformer's attention mechanism** is mathematically a fully-connected graph where every token attends to every other token — attention weights are edge weights.
- **Knowledge graphs** (used in Google Search, enterprise RAG, and structured LLM grounding) are literal graphs where entities are nodes and relationships are edges.
- **Computation graphs** in every deep learning framework (PyTorch's autograd, TensorFlow's graph mode) are DAGs of operations.
- **Reasoning-over-graphs** is central to how systems like GraphRAG (Microsoft) improve on vector-only RAG by capturing structured relationships between documents.

2.9 Tries (Prefix Trees)

What it is

A tree where each node represents a character, and paths from root to nodes represent prefixes of stored strings.

Complexity

Operation	Time
Insert word of length L	O(L)
Search word of length L	O(L)
Prefix search	O(L)

When to reach for it

- Autocomplete
- Spell check
- IP routing (longest prefix match)
- Word games (Boggle, Scrabble)
- Dictionary problems

Where it shows up in AI/ML

- **BPE tokenizers** (used in GPT, Llama, Claude) use trie-like structures internally to match the longest known subword at each position.
- **Constrained decoding** for LLMs (forcing output to match a grammar or schema) uses tries to represent the allowed set of continuations at each step.

2.10 Union-Find (Disjoint Set Union / DSU)

What it is

A structure that tracks a set of elements partitioned into disjoint subsets, with two operations: - `find(x)` : which subset does x belong to? - `union(x, y)` : merge the subsets containing x and y

With **path compression + union by rank**, both operations are nearly O(1) amortized.

When to reach for it

- Detecting cycles in undirected graphs
- Kruskal's algorithm (minimum spanning tree)

- Connected components
- "Number of islands" style problems (alternative to BFS/DFS)
- Account/friend merging problems

Where it shows up in AI/ML

- **Clustering algorithms** (particularly hierarchical clustering) use Union-Find internally.
- **Connected component analysis** in image segmentation (CNN post-processing) uses it.
- **Merging duplicate entities** in knowledge graph construction relies on it.

Reference implementation

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]] # path compression
            x = self.parent[x]
        return x

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        return True
```

PART III — ALGORITHM PATTERNS

If Part II gave you the *containers*, Part III gives you the *techniques*. Almost every interview problem is a variation on 10–12 core patterns. Learn them by name and 80% of problems become "oh, this is a sliding window problem."

3.1 Two Pointers

The idea

Use two indices moving through the data — often from opposite ends toward each other, or one fast and one slow.

When it applies

- Sorted arrays where you need to find pairs/triplets with some property
- Palindromes
- Removing duplicates in-place
- Container/water problems
- Merging sorted structures

Template

```
def two_pointers(arr):
    left, right = 0, len(arr) - 1
    while left < right:
        if condition_met(arr[left], arr[right]):
            # do something
            left += 1
            right -= 1
        elif need_larger_sum:
            left += 1
        else:
            right -= 1
```

Real-world resonance

Two pointers is how you'd implement a merge step in a merge-sort of two databases, or how you'd walk two sorted logs to find matching entries — a common ETL task.

3.2 Sliding Window

The idea

Maintain a "window" `[left, right]` over the data. Expand `right` to include new elements; shrink from `left` when a constraint is violated. Each element is visited at most twice → $O(n)$.

When it applies

- Longest/shortest substring or subarray with property X
- Fixed-size window statistics (max in window, average)
- Anagram / permutation matching in strings

Templates

Variable-size window:

```
def sliding_window(s):
    left = 0
    state = initial_state()
    best = 0
    for right in range(len(s)):
        update_state_add(state, s[right])
        while constraint_violated(state):
            update_state_remove(state, s[left])
            left += 1
        best = max(best, right - left + 1)
    return best
```

Fixed-size window:

```
def fixed_window(arr, k):
    window_sum = sum(arr[:k])
    best = window_sum
    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i - k]
        best = max(best, window_sum)
    return best
```

Real-world resonance

Rate limiters, real-time analytics dashboards, monitoring "requests in the last 5 minutes" — all sliding windows.

3.3 Binary Search

The idea

Repeatedly halve the search space by comparing to the midpoint. Requires sorted data (or a monotonic property).

When it applies

- Searching sorted arrays
- Finding boundaries ("first true" / "last false" patterns)
- Search-on-answer problems ("what's the smallest X such that condition C(X) is true?")
- Search in rotated sorted arrays
- Any log n requirement

Templates

Standard search:

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Find first index where condition is true (my favorite):

```
def first_true(lo, hi, condition):
    while lo < hi:
        mid = (lo + hi) // 2
        if condition(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Real-world resonance

Database indexes (B-trees), git bisect for finding the commit that broke things, hyperparameter tuning, model performance thresholds.

3.4 Recursion & Backtracking

The idea

Solve a big problem by expressing it in terms of smaller versions of itself. Backtracking adds "try, then undo if it doesn't work" for exploring combinatorial spaces.

When it applies

- Trees (naturally recursive)
- Permutations, combinations, subsets
- Puzzles: N-Queens, Sudoku, word search
- Divide-and-conquer problems

Template

```
def backtrack(state, choices):
    if is_solution(state):
        results.append(state.copy())
        return
    for choice in choices:
        if is_valid(state, choice):
            state.append(choice)      # make choice
            backtrack(state, choices)
            state.pop()              # undo choice
```

Real-world resonance

SAT solvers, constraint satisfaction (scheduling, resource allocation), compiler optimization, robot path planning, and — crucially — **reasoning chains in LLMs**: modern agentic systems use backtracking-like exploration when a plan fails.

3.5 Dynamic Programming (DP)

The idea

Break a problem into overlapping subproblems, solve each subproblem once, and cache the result. Two flavors: - **Top-down (memoization)**: recursion + a cache. - **Bottom-up (tabulation)**: iteratively fill a table from base cases.

When it applies

The problem has both: 1. **Optimal substructure**: the answer to the big problem uses answers to smaller versions. 2. **Overlapping subproblems**: the same sub-answers get needed multiple times.

Common DP problems: Fibonacci, coin change, longest common subsequence, knapsack, edit distance, house robber, unique paths.

Templates

Top-down (memoization):

```
from functools import lru_cache

@lru_cache(maxsize=None)
def solve(state):
    if is_base_case(state):
        return base_value
    return combine(solve(smaller_state_1), solve(smaller_state_2))
```

Bottom-up (tabulation):

```
def solve(n):
    dp = [0] * (n + 1)
    dp[0] = base_value
    for i in range(1, n + 1):
        dp[i] = f(dp[i-1], dp[i-2], ...)
    return dp[n]
```

Real-world resonance

- **Sequence alignment in bioinformatics** (BLAST, Smith-Waterman) is DP.
- **Edit distance / Levenshtein** underlies spell check, autocorrect, and diff tools.
- **LLM decoding with KV-cache** is a DP-style optimization — you cache the key/value tensors from previous tokens instead of recomputing them.
- **Reinforcement learning value iteration** is DP over states.

3.6 Greedy Algorithms

The idea

At each step, make the locally optimal choice, hoping it leads to a global optimum. Simpler than DP but only works when the problem has the *greedy choice property*.

When it applies

- Interval scheduling (activity selection)
- Huffman coding
- Kruskal's / Prim's for minimum spanning trees
- Jump game
- Gas station problems

The catch

Greedy is deceptive — it often *looks* right but *isn't*. Coin change with arbitrary denominations, for example, needs DP. Always verify greedy works with a proof or a strong intuition ("we can never gain by delaying").

3.7 BFS & DFS on Graphs/Trees

BFS (Breadth-First Search) — uses a queue

```
from collections import deque

def bfs(start, graph):
    visited = {start}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        # process node
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

DFS (Depth-First Search) — uses recursion or a stack

```
def dfs(node, graph, visited):
    if node in visited: return
    visited.add(node)
    # process node
    for neighbor in graph[node]:
        dfs(neighbor, graph, visited)
```

When to use which

- **BFS**: shortest path in unweighted graph, level-by-level exploration, "spread" problems (rotting oranges, distance from source).
- **DFS**: exploring all paths, detecting cycles, topological sort, connected components, tree recursions.

3.8 Sorting Fundamentals

You don't need to implement quicksort in interviews — Python's `sorted()` (Timsort, $O(n \log n)$) is nearly always correct. But you *do* need to know:

- **Comparison-based sorts are $O(n \log n)$ lower bound** — you can't sort n numbers faster with comparisons alone.
- **Counting sort / radix sort** achieve $O(n)$ but only for integers in a bounded range.
- **Sorting is often the first step** of an efficient algorithm (interval problems, greedy problems).
- `key=` and `functools.cmp_to_key` are your friends for custom comparisons.

```
# Sort by multiple keys
arr.sort(key=lambda x: (x.age, -x.salary)) # age asc, salary desc

# Custom comparator
from functools import cmp_to_key
arr.sort(key=cmp_to_key(lambda a, b: a - b))
```

3.9 Bit Manipulation

Core operations

Operation	Result
<code>a & b</code>	AND — bits set in both
<code>a b</code>	OR — bits set in either
<code>a ^ b</code>	XOR — bits set in exactly one
<code>~a</code>	NOT
<code>a << k</code>	left shift (multiply by 2^k)
<code>a >> k</code>	right shift (divide by 2^k)

Killer tricks

```
x & (x - 1)      # clears the lowest set bit
x & -x           # isolates the lowest set bit
bin(x).count('1') # count set bits (or bit_count() in Python 3.10+)
x ^ x           # 0 (XOR self is zero – used in "find the unique number")
```

Where it shows up

- Bitmask DP (traveling salesman for small n)
- Permission flags
- Fast subset enumeration
- Model quantization in ML (INT8, INT4 inference).

3.10 Divide & Conquer

Break the problem into independent subproblems, solve recursively, combine. Merge sort, quicksort, binary search, and Strassen's matrix multiplication are examples.

Distinct from DP because subproblems *don't* overlap — no caching needed.

PART IV — PYTHON POWER TOOLS FOR INTERVIEWS

Python's standard library gives you superpowers if you know where to look. These are the modules and idioms that turn 20-line solutions into 5-line ones.

4.1 collections

```
from collections import Counter, defaultdict, deque, OrderedDict

Counter("hello")           # frequency count
Counter(a) - Counter(b)    # multiset difference
defaultdict(list)          # auto-init missing keys
deque(maxlen=k)            # bounded double-ended queue
```

4.2 heapq

```
import heapq
heapq.heappush(h, x); heapq.heappop(h)
heapq.nlargest(k, iterable, key=...)
heapq.heapify(list)        # in-place, O(n)
```

4.3 bisect — Binary Search on Sorted Lists

```
import bisect
bisect.bisect_left(sorted_arr, x) # leftmost insertion point
bisect.bisect_right(sorted_arr, x) # rightmost insertion point
bisect.insort(sorted_arr, x)      # insert while maintaining sort
```

4.4 itertools

```
from itertools import combinations, permutations, product, accumulate, groupby

list(combinations([1,2,3], 2)) # [(1,2),(1,3),(2,3)]
list(permutations([1,2,3], 2)) # ordered pairs
list(product([0,1], repeat=3)) # cartesian product
list(accumulate([1,2,3,4]))    # [1,3,6,10] - running sum (prefix sums!)
```

4.5 functools

```
from functools import lru_cache, reduce, cmp_to_key

@lru_cache(maxsize=None) # instant memoization
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)

reduce(lambda a,b: a*b, [1,2,3,4]) # 24
```

4.6 String Tricks

```
s.isdigit(), s.isalpha(), s.isalnum()
ord('a')      # 97 - char to int
chr(97)       # 'a'
'a'.lower()
sorted("bca") # ['a','b','c']
```

4.7 The Assignment / Swap Idiom

```
a, b = b, a # swap without temp
a, b, c = 1, 2, 3 # multiple assignment
first, *middle, last = [1,2,3,4,5] # unpacking
```

4.8 One-Liner Superpowers

```
# Transpose a matrix
list(zip(*matrix))

# Flatten a 2D list
flat = [x for row in matrix for x in row]

# Reverse a string / list
s[::-1]

# Prefix sums
from itertools import accumulate
prefix = list(accumulate(nums))
```

PART V — OBJECT-ORIENTED PROGRAMMING & DESIGN PATTERNS

Even LeetCode-style problems increasingly ask you to *design* something — an LRU cache, a rate limiter, a parking lot, a Twitter feed. This is where OOP shows up in interviews.

5.1 The Four Pillars

1. Encapsulation — Bundle data + methods, hide internals.

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance # convention: _ means "private-ish"

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def get_balance(self):
        return self._balance
```

2. Inheritance — A child class reuses/extends a parent class.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self): raise NotImplementedError

class Dog(Animal):
    def speak(self): return f"{self.name} says Woof"

class Cat(Animal):
    def speak(self): return f"{self.name} says Meow"
```

3. Polymorphism — Same interface, different behaviors.

```
animals = [Dog("Rex"), Cat("Whiskers")]
for a in animals: print(a.speak()) # each behaves per its type
```

4. Abstraction — Expose what, hide how. Use `abc` module for enforcement.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): ...
```

5.2 SOLID Principles (interview gold)

- **Single Responsibility**: a class does one thing well.
- **Open/Closed**: open for extension, closed for modification.
- **Liskov Substitution**: subclasses must be usable in place of parents.
- **Interface Segregation**: many small interfaces beat one fat one.
- **Dependency Inversion**: depend on abstractions, not concretions.

5.3 Must-Know Design Patterns

Singleton (one instance globally)

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Factory (create objects without specifying exact class)

```
class ShapeFactory:
    @staticmethod
    def create(kind):
        return {"circle": Circle, "square": Square}[kind]()
```

Observer (pub-sub)

```
class Subject:
    def __init__(self):
        self.observers = []
    def subscribe(self, obs): self.observers.append(obs)
    def notify(self, event):
        for obs in self.observers: obs.update(event)
```

Strategy (swap algorithms at runtime)

```
class Sorter:
    def __init__(self, strategy):
        self.strategy = strategy
    def sort(self, data):
        return self.strategy(data)
```

Decorator (Python has native support!)

```
def log_calls(fn):
    def wrapper(*args, **kwargs):
        print(f"Calling {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper

@log_calls
def greet(name): return f"Hi {name}"
```

5.4 Python OOP Specials

```
class Point:
    __slots__ = ('x', 'y')          # faster, less memory (no __dict__)

    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):             # dev-friendly string
        return f"Point({self.x}, {self.y})"

    def __eq__(self, other):        # ==
        return self.x == other.x and self.y == other.y

    def __hash__(self):            # allows use in sets/dicts
        return hash((self.x, self.y))

    def __lt__(self, other):        # < (needed for sorting/heaps)
        return (self.x, self.y) < (other.x, other.y)

    @classmethod
    def origin(cls): return cls(0, 0)

    @staticmethod
    def distance(a, b):
        return ((a.x-b.x)**2 + (a.y-b.y)**2) ** 0.5
```

PART VI — SYSTEM DESIGN PRIMER

For senior interviews, system design matters as much as coding. This section gets you 80% of the way for both algorithmic-design problems ("design LRU cache") and higher-level system design rounds ("design Twitter").

6.1 The Universal Framework (use this for every system design question)

- 1. **Clarify requirements** (5 min)
- 2. Functional: what does it do?
- 3. Non-functional: scale? latency? consistency? availability?
- 4. Ask: "How many users? How many QPS? Read-heavy or write-heavy?"
- 5. **Estimate scale** (2 min)
- 6. QPS, storage per day/year, bandwidth.
- 7. Rule of thumb: $100M\text{ DAU} \times 10\text{ actions/day} = 1B\text{ events/day} \approx 12k\text{ QPS average}$.
- 8. **High-level design** (10 min)
- 9. Draw boxes: clients → load balancer → app servers → cache → database.
- 10. Name the components; don't sweat details yet.
- 11. **Deep dive into 1-2 key components** (15 min)
- 12. Data model / schema.
- 13. The trickiest algorithm.
- 14. Failure modes.
- 15. **Address bottlenecks & scale** (5 min)
- 16. Where does it break at 10x scale?
- 17. Sharding, caching layers, async processing, CDNs.

6.2 Building Blocks You Must Know

Component	What it does	When to use
Load balancer	Distributes requests across servers	Any multi-server system
Cache (Redis, Memcached)	In-memory KV store	Read-heavy workloads, sessions
CDN (CloudFlare, CloudFront)	Edge cache for static assets	Global users, images/videos
SQL DB (PostgreSQL, MySQL)	ACID, joins, structured	Financial data, relational data
NoSQL KV (DynamoDB)	Massive scale, simple queries	Sessions, user profiles
NoSQL Document (MongoDB)	Flexible schema	Product catalogs, CMS
Wide-column (Cassandra)	Time-series, write-heavy	Logs, metrics, activity feeds
Message queue (Kafka, SQS)	Async work, decoupling	Order processing, notifications
Search (Elasticsearch)	Full-text search	Product search, log search
Vector DB (Pinecone, Weaviate)	Semantic similarity	RAG, recommendations
Object storage (S3)	Cheap file storage	Images, backups, ML datasets

6.3 Key Concepts

CAP Theorem: In a partitioned network, you must trade Consistency vs. Availability. Bank transactions → CP. Social feeds → AP.

Consistency models: strong, eventual, causal. Most large-scale systems accept eventual consistency.

Caching strategies: cache-aside (read-through), write-through, write-behind. Know cache invalidation is the second-hardest problem in CS (the first is naming things).

Sharding: partition data across machines by key (hash-based) or range. Avoids single-machine bottleneck at the cost of complex queries.

Replication: primary-replica for read scaling; multi-primary for write scaling (but with conflict headaches).

Rate limiting: token bucket, leaky bucket, sliding window. Every public API needs one.

Idempotency: designing operations so retries are safe. Critical for distributed systems.

6.4 Classic Design Problems to Practice

- Design a URL shortener (bit.ly)
 - Design an LRU cache
 - Design a rate limiter
 - Design Twitter's newsfeed
 - Design WhatsApp / Slack
 - Design a search autocomplete
 - Design a distributed key-value store
 - Design a video streaming service (Netflix)
 - Design a ride-sharing service (Uber)
 - Design a distributed job scheduler
 - Design a recommendation system
 - **Design a RAG system** (increasingly common — chunking + embedding + retrieval + reranking + LLM)
-

PART VII — INTERVIEW META-SKILLS

Coding well isn't enough. These are the "how you present the answer" skills that determine hire vs. no-hire.

7.1 The Universal Problem-Solving Framework

For every problem, in this order:

1. **Restate the problem** — "So we're given X, and I need to return Y. Is that right?"
2. **Ask clarifying questions** — Empty input? Duplicates allowed? Negative numbers? Multiple valid answers? What's the expected size?
3. **Walk through an example by hand** — Both the given example and one edge case you invent.
4. **Propose the naive solution** — State its complexity. It's ok! "Here's the $O(n^2)$ brute force. Let me see if we can do better."
5. **Identify the pattern / improve** — "The nested loop is checking pairs — that's screaming hashmap."
6. **Code the solution** — Talk while coding. Silence is the enemy.
7. **Trace through your code** with the example.
8. **Analyze complexity** — Time and space, best/average/worst if they differ.
9. **Discuss edge cases and tradeoffs** — What breaks it? What would you change at 10x scale?

7.2 Complexity Analysis Reference

Big-O	Name	Example
$O(1)$	Constant	Dict lookup, array access
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Single loop
$O(n \log n)$	Linearithmic	Sorting, divide & conquer
$O(n^2)$	Quadratic	Nested loops (pair sums)
$O(2^n)$	Exponential	Subsets, brute force recursion
$O(n!)$	Factorial	Permutations

For $n = 10^6$: $O(n)$ and $O(n \log n)$ are fine. $O(n^2)$ is not. For $n = 10^3$: $O(n^2)$ is fine. For $n = 20$: $O(2^n)$ is fine.

7.3 Space Complexity — don't forget

- Recursion uses $O(\text{depth})$ stack space.
- A hashmap with n entries uses $O(n)$ space.
- In-place algorithms use $O(1)$ auxiliary space (Python slicing is *not* in-place).

7.4 Common Edge Cases to Always Consider

- Empty input `[]`, `""`, `None`
- Single element `[x]`
- Duplicates
- Negative numbers, zero
- Very large input (integer overflow doesn't apply in Python, but time does)
- Sorted / reverse-sorted input
- All-same input `[5,5,5,5]`

7.5 Talking Out Loud — a template

"So I have this array and I need to find two numbers summing to target. The brute force is nested loops — $O(n^2)$. But every time I look at `nums[i]`, I'm really asking: 'have I seen `target - nums[i]`?' That's an $O(1)$ hashmap question. So

| one pass, tracking what I've seen, checking the complement each step. That gets us $O(n)$ time, $O(n)$ space."

Practice this. It's the difference between "candidate solved the problem" and "candidate solved the problem *cleanly* — hire."

PART VIII — THE PROBLEM SET

How to read each problem: Level (Easy / Medium / Hard) → Problem statement → Example → Hint(s) → Pattern tag. Solutions are in Part IX — **do not peek** until you've written code.

Tier progression: Easy problems come first in each category (foundations). Then medium (real interview level). Then a few hard (senior interviews / bonus). Move through categories in order the first time; later, mix them.

Category A — Arrays & Lists (20 problems)

A1. Two Sum — Easy

Given an array of integers `nums` and integer `target`, return indices of the two numbers that add up to `target`. Example: `nums=[2,7,11,15]`, `target=9` → `[0,1]` Hint: You want $O(1)$ lookup of "have I seen the complement?" — reach for a hashmap. Pattern: Hashmap for lookup.

A2. Best Time to Buy and Sell Stock — Easy

Given daily stock prices, find the max profit from one buy + one sell. Example: `[7,1,5,3,6,4]` → 5 (buy at 1, sell at 6). Hint: Track the minimum price seen so far and the best profit at each step. Pattern: Single-pass tracking.

A3. Contains Duplicate — Easy

Return `True` if any value appears at least twice. Example: `[1,2,3,1]` → `True` Hint: Compare `len(nums)` to `len(set(nums))`. Pattern: Set for uniqueness.

A4. Maximum Subarray — Easy

Find the contiguous subarray with the largest sum. Example: `[-2,1,-3,4,-1,2,1,-5,4]` → 6 Hint: At each element, decide: extend the current subarray or restart from here? Pattern: Kadane's algorithm / DP.

A5. Product of Array Except Self — Medium

Return an array where `output[i]` = product of all elements except `nums[i]`. No division. Example: `[1,2,3,4]` → `[24,12,8,6]` Hint: Two passes. First pass: prefix products. Second pass: suffix products, multiplying in. Pattern: Prefix + suffix arrays.

A6. Maximum Product Subarray — Medium

Find the contiguous subarray with the largest product. Example: `[2,3,-2,4]` → 6 Hint: Track *both* max and min ending here (because a negative can flip min into max). Pattern: DP with two-state tracking.

A7. Find Minimum in Rotated Sorted Array — Medium

A sorted array was rotated at some pivot. Find the minimum. Example: `[4,5,6,7,0,1,2]` → 0 Hint: Binary search. The half containing the minimum is the unsorted half. Pattern: Modified binary search.

A8. Search in Rotated Sorted Array — Medium

Find target index in a rotated sorted array, or -1. Example: `nums=[4,5,6,7,0,1,2]`, `target=0` → 4 Hint: Binary search. Determine which half is sorted, then check if target is in it. Pattern: Modified binary search.

A9. Merge Intervals — Medium

Merge all overlapping intervals. Example: `[[1,3],[2,6],[8,10],[15,18]]` → `[[1,6],[8,10],[15,18]]` Hint: Sort by start. Walk through; if current overlaps last merged, extend end. Pattern: Sort-then-sweep.

A10. Insert Interval — Medium

Insert a new interval into a sorted, non-overlapping list, merging as needed. Example: `intervals=[[1,3],[6,9]]`, `new=[2,5]` → `[[1,5],[6,9]]` Hint: Three phases — before, overlapping (merge), after. Pattern: Interval merging.

A11. Non-Overlapping Intervals — Medium

Given intervals, find minimum number to remove so the rest don't overlap. Example: `[[1,2],[2,3],[3,4],[1,3]]` → `1` Hint: Sort by *end* time and greedily keep the earliest-ending non-overlapping ones. Pattern: Greedy interval scheduling.

A12. Rotate Array — Medium

Rotate array to the right by *k* steps. Example: `[1,2,3,4,5,6,7]`, `k=3` → `[5,6,7,1,2,3,4]` Hint: Reverse whole array, then reverse first *k*, then reverse rest. Pattern: Reversal trick.

A13. Move Zeroes — Easy

Move all 0s to the end, preserving order of non-zeros, in-place. Example: `[0,1,0,3,12]` → `[1,3,12,0,0]` Hint: Two pointers — one for reading, one for writing. Pattern: Two pointers (fast/slow write).

A14. Container With Most Water — Medium

Given heights, find two lines forming the container with the most water. Example: `[1,8,6,2,5,4,8,3,7]` → `49` Hint: Two pointers at the ends. Move the shorter one inward. Pattern: Two pointers converging.

A15. 3Sum — Medium

Find all unique triplets that sum to 0. Example: `[-1,0,1,2,-1,-4]` → `[[-1,-1,2],[-1,0,1]]` Hint: Sort. For each *i*, use two pointers on the rest. Skip duplicates. Pattern: Sorted + two pointers.

A16. Trapping Rain Water — Hard

Given bar heights, compute total trapped rainwater. Example: `[0,1,0,2,1,0,1,3,2,1,2,1]` → `6` Hint: Water at position *i* = `min(max_left, max_right) - height[i]`. Two-pointer approach can do it in *O*(1) space. Pattern: Two pointers / prefix arrays.

A17. Subarray Sum Equals K — Medium

Number of contiguous subarrays with sum = *k*. Example: `nums=[1,1,1]`, `k=2` → `2` Hint: Prefix sums + hashmap. If `prefix_sum - k` has been seen, we have a valid subarray. Pattern: Prefix sum + hashmap.

A18. Find All Duplicates in an Array — Medium

$1 \leq a[i] \leq n$. Find all elements appearing twice, in *O*(1) extra space. Example: `[4,3,2,7,8,2,3,1]` → `[2,3]` Hint: Use the array itself as a hashmap — negate `nums[abs(v)-1]` as you scan. Pattern: Index-as-hashmap trick.

A19. Next Permutation — Medium

Rearrange numbers to the next lexicographically greater permutation. In-place. Example: `[1,2,3]` → `[1,3,2]` Hint: Find the first descending element from the right; swap with the smallest element to its right that's larger; reverse the suffix. Pattern: Algorithmic construction.

A20. Median of Two Sorted Arrays — Hard

Find the median of two sorted arrays in *O*(log(min(*m*,*n*))). Example: `[1,3]`, `[2]` → `2.0` Hint: Binary search on the smaller array — partition both so left halves and right halves are balanced. Pattern: Binary search on partitions.

Category B — Strings (15 problems)

B1. Valid Palindrome — Easy

Ignoring non-alphanumerics and case, is the string a palindrome? Example: `"A man, a plan, a canal: Panama"` → `True` Hint: Two pointers from both ends, skipping non-alphanumeric. Pattern: Two pointers.

B2. Valid Anagram — Easy

Are two strings anagrams? Example: `"listen"`, `"silent"` → `True` Hint: `Counter(a) == Counter(b)`. Pattern: Frequency counting.

B3. Longest Common Prefix — Easy

Longest prefix shared by all strings in an array. Example: `["flower", "flow", "flight"]` → `"fl"` Hint: Zip the strings and take chars while all match. Pattern: Column-wise comparison.

B4. Reverse Words in a String — Medium

Reverse word order, trim whitespace, single-space between words. Example: `" hello world "` → `"world hello"` Hint: `" .join(s.split()[::-1])` in Python — but know the manual approach too. Pattern: Split-reverse-join.

B5. Longest Substring Without Repeating Characters — Medium

Length of the longest substring with all unique characters. Example: `"abcabcbb"` → `3` Hint: Sliding window. Track chars in current window in a set. Pattern: Sliding window.

B6. Longest Repeating Character Replacement — Medium

You can replace up to `k` chars. Return length of longest same-char substring possible. Example: `s="AABABBA", k=1` → `4` Hint: Sliding window. Window is valid if `window_size - max_freq_in_window <= k`. Pattern: Sliding window with frequency.

B7. Minimum Window Substring — Hard

Shortest substring of `s` containing all chars of `t`. Example: `s="ADOBECODEBANC", t="ABC"` → `"BANC"` Hint: Sliding window. Expand right until valid, then shrink left while still valid, track min. Pattern: Sliding window with counts.

B8. Group Anagrams — Medium

Group strings that are anagrams of each other. Example: `["eat", "tea", "tan", "ate", "nat", "bat"]` → `[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]` Hint: Use sorted-string as the hashmap key. Pattern: Hashmap keyed by signature.

B9. Valid Parentheses — Easy

Check if `()[]{}` are balanced and properly nested. Example: `"()[]{}"` → `True` Hint: Stack. Push opens, pop on close and check match. Pattern: Stack.

B10. Generate Parentheses — Medium

Generate all valid combinations of `n` pairs of parentheses. Example: `n=3` → `[["((()))", "(()())", "(())()", "()(())", "()()()"]]` Hint: Backtracking. Only add "(" if `open < n`, only add ")" if `close < open`. Pattern: Backtracking with constraints.

B11. String to Integer (atoi) — Medium

Parse a string into an int (handle sign, whitespace, overflow). Example: `" -42abc"` → `-42` Hint: State machine — skip whitespace, read sign, read digits, clamp. Pattern: Careful parsing.

B12. Longest Palindromic Substring — Medium

Longest substring of `s` that is a palindrome. Example: `"babad"` → `"bab"` or `"aba"` Hint: Expand-around-center for each index (odd and even). Pattern: Expand around center.

B13. Palindromic Substrings — Medium

Count all palindromic substrings. Example: `"aaa"` → `6` Hint: Same expand-around-center; increment a counter for each expansion. Pattern: Expand around center.

B14. Encode and Decode Strings — Medium

Design encode/decode for a list of strings (they may contain any character). Hint: Prefix each string with its length + a delimiter. Decode: read length, then that many chars. Pattern: Length-prefixed encoding.

B15. Word Break — Medium

Can `s` be segmented into a space-separated sequence of dictionary words? Example: `s="leetcode", dict=["leet", "code"]` → `True` Hint: DP. `dp[i] = can s[:i] be segmented?` Pattern: 1D DP.

Category C — Hashmaps & Sets (15 problems)

C1. First Unique Character in a String — Easy

Index of first non-repeating character, or -1. Example: "leetcode" → 0 Hint: Count frequencies, then scan for first count == 1. Pattern: Two-pass counting.

C2. Ransom Note — Easy

Can ransomNote be constructed from letters in magazine? Example: note="aa", mag="aab" → True Hint: Counter(note) - Counter(mag) should be empty. Pattern: Counter subtraction.

C3. Isomorphic Strings — Easy

Can chars in s be mapped 1-to-1 to chars in t to produce t? Example: "egg", "add" → True Hint: Two hashmaps (or one, mapping both directions). Pattern: Bidirectional mapping.

C4. Word Pattern — Easy

Pattern of chars maps to words in a sentence? Example: pattern="abba", s="dog cat cat dog" → True Hint: Same as isomorphic — two-way mapping between chars and words. Pattern: Bidirectional mapping.

C5. Happy Number — Easy

Repeatedly replace n with sum of squares of digits. Does it reach 1? Example: 19 → True ($1^2+9^2=82$, $8^2+2^2=68$, $6^2+8^2=100$, $1^2+0^2+0^2=1$) Hint: Detect cycles using a set (or fast/slow pointers). Pattern: Cycle detection.

C6. Longest Consecutive Sequence — Medium

Longest streak of consecutive integers in an unsorted array. O(n). Example: [100,4,200,1,3,2] → 4 (1,2,3,4) Hint: Put in a set. For each number that has no predecessor in the set, count the streak. Pattern: Set + "start of streak" trick.

C7. Top K Frequent Elements — Medium

Return the k most frequent elements. Example: nums=[1,1,1,2,2,3], k=2 → [1,2] Hint: Counter + heap (or Counter.most_common(k)). Bucket sort is O(n). Pattern: Frequency + heap / bucket sort.

C8. Subarray Sum Equals K — Medium

(Duplicate of A17, revisited under hashmap lens.) Hint: Prefix sums; hashmap of "prefix sum seen count." Pattern: Prefix sum + hashmap.

C9. Longest Substring with At Most K Distinct Characters — Medium

Example: s="eceba", k=2 → 3 ("ece") Hint: Sliding window with hashmap counting distinct chars. Pattern: Sliding window + hashmap.

C10. Two Sum II - Sorted Array — Easy

Same as Two Sum but array is sorted; solve in O(1) extra space. Hint: Two pointers, not a hashmap. Pattern: Two pointers.

C11. Group Shifted Strings — Medium

Group strings that are shifts of each other (e.g., "abc", "bcd", "xyz"). Hint: Signature = differences between consecutive chars. Pattern: Hashmap keyed by signature.

C12. Design HashMap — Easy

Implement a HashMap without built-in hashmap. Hint: Array of linked lists (chaining) or open addressing. Pattern: Data-structure design.

C13. Random Pick with Weight — Medium

Given weights, pick an index with probability proportional to weight. Hint: Prefix sums + binary search on random value in [0, total). Pattern: Prefix sum + binary search.

C14. Insert Delete GetRandom O(1) — Medium

Design a structure with O(1) insert, delete, and random-element. Hint: List for O(1) random; hashmap of value→index for O(1) delete (swap with last). Pattern: List + hashmap combo.

C15. LRU Cache — Medium

Design an LRU cache with O(1) `get` and `put`. Hint: Doubly linked list (for order) + hashmap (for O(1) node lookup). Or `OrderedDict`. Pattern: Linked list + hashmap.

Category D — Two Pointers & Sliding Window (12 problems)

D1. Reverse String — Easy

Reverse in-place. Hint: Two pointers converging. Pattern: Two pointers.

D2. Squares of a Sorted Array — Easy

Return squares of a sorted (possibly negative) array, sorted. Example: `[-4, -1, 0, 3, 10] → [0, 1, 9, 16, 100]` Hint: Two pointers from both ends; largest square comes from one end. Pattern: Two pointers (fill from back).

D3. Remove Duplicates from Sorted Array — Easy

In-place; return new length. Hint: Slow/fast pointers. Slow points to next write, fast reads. Pattern: Two pointers (fast/slow).

D4. Remove Element — Easy

Remove all occurrences of `val`, in-place. Hint: Same two-pointer pattern as D3. Pattern: Two pointers.

D5. Sort Colors — Medium

Sort array of 0s, 1s, 2s in-place (Dutch national flag). Example: `[2, 0, 2, 1, 1, 0] → [0, 0, 1, 1, 2, 2]` Hint: Three pointers — low (0s boundary), mid (current), high (2s boundary). Pattern: Three-way partition.

D6. Maximum Average Subarray I — Easy

Max average of any contiguous subarray of size k. Hint: Fixed-size sliding window. Pattern: Fixed sliding window.

D7. Longest Substring with Same Letter after k Replacements — Medium

(Same as B6.)

D8. Fruit Into Baskets — Medium

Longest subarray with at most 2 distinct values. Hint: Sliding window with hashmap; shrink when >2 distinct. Pattern: Sliding window.

D9. Permutation in String — Medium

Does `s2` contain a permutation of `s1`? Hint: Fixed-size window; compare Counter windows. Pattern: Sliding window + counter equality.

D10. Find All Anagrams in a String — Medium

Return all start indices of anagrams of `p` in `s`. Hint: Same as D9 but collect all start indices. Pattern: Sliding window + counter equality.

D11. Subarrays with Product Less Than K — Medium

Count contiguous subarrays whose product $< k$. Hint: Sliding window; for each right, count = right - left + 1. Pattern: Sliding window.

D12. Minimum Size Subarray Sum — Medium

Smallest length of contiguous subarray with sum $\geq$ target. Hint: Sliding window; expand right, shrink left while sum $\geq$ target. Pattern: Sliding window (shrinkable).

Category E — Stacks & Queues (12 problems)

E1. Valid Parentheses — Easy

(Duplicate of B9.) Pattern: Stack.

E2. Min Stack — Medium

Design a stack with $O(1)$ push, pop, top, and getMin. Hint: Auxiliary stack tracking min so far. Pattern: Stack + auxiliary stack.

E3. Evaluate Reverse Polish Notation — Medium

Evaluate expressions in postfix notation. Example: ["2", "1", "+", "3", "*"] $\rightarrow$ 9 Hint: Push numbers; on operator, pop two and push result. Pattern: Stack.

E4. Daily Temperatures — Medium

For each day, how many days until a warmer day? Example: [73, 74, 75, 71, 69, 72, 76, 73] $\rightarrow$ [1, 1, 4, 2, 1, 1, 0, 0] Hint: Monotonic decreasing stack of indices. Pattern: Monotonic stack.

E5. Next Greater Element — Medium

For each element, find next greater to its right; -1 if none. Hint: Monotonic stack, scan right to left (or left to right). Pattern: Monotonic stack.

E6. Largest Rectangle in Histogram — Hard

Largest rectangle area in a histogram. Example: [2, 1, 5, 6, 2, 3] $\rightarrow$ 10 Hint: Monotonic increasing stack. On each pop, compute area with popped bar as smallest. Pattern: Monotonic stack.

E7. Sliding Window Maximum — Hard

Max in every window of size k. Example: nums=[1, 3, -1, -3, 5, 3, 6, 7], k=3 $\rightarrow$ [3, 3, 5, 5, 6, 7] Hint: Monotonic decreasing deque of indices. Pattern: Monotonic deque.

E8. Implement Queue Using Stacks — Easy

Hint: Two stacks — one for enqueue, one for dequeue. Pattern: Two stacks.

E9. Implement Stack Using Queues — Easy

Hint: One queue; on push, rotate to bring new element to front. Pattern: Queue manipulation.

E10. Decode String — Medium

"3[a2[c]]" $\rightarrow$ "accaccacc". Hint: Stack of (count, previous string) tuples. Pattern: Stack with state.

E11. Asteroid Collision — Medium

Simulate asteroids of $\pm$ sizes colliding. Example: [5, 10, -5] $\rightarrow$ [5, 10] Hint: Stack; on negative, pop while smaller collides with

it. Pattern: Stack simulation.

E12. Basic Calculator — Hard

Evaluate `"1 + (2 - (3 + 4))"`. Hint: Stack of signs / parentheses state. Or recursive parse. Pattern: Stack for parsing.

Category F — Linked Lists (12 problems)

F1. Reverse Linked List — Easy

Iterative and recursive. Hint: Three pointers: `prev`, `curr`, `next`. Pattern: Pointer reversal.

F2. Detect Cycle — Easy

Does the list have a cycle? Hint: Floyd's tortoise and hare — slow moves 1, fast moves 2. Pattern: Fast/slow pointers.

F3. Find Cycle Start — Medium

Return the node where the cycle begins. Hint: After they meet, reset one pointer to head and step both by 1. Pattern: Fast/slow + math trick.

F4. Merge Two Sorted Lists — Easy

Merge into one sorted list. Hint: Dummy head; walk both lists picking the smaller. Pattern: Merge with dummy node.

F5. Merge K Sorted Lists — Hard

Merge k sorted linked lists. Hint: Min-heap of (val, list_index, node), or divide-and-conquer pairwise merge. Pattern: Heap merge.

F6. Remove Nth Node from End — Medium

Remove the nth node from the end in one pass. Hint: Two pointers, n apart, walk together. Pattern: Two pointers (gap of n).

F7. Reorder List — Medium

`L0→L1→...→Ln-1→Ln` becomes `L0→Ln→L1→Ln-1→...`. Hint: Find middle, reverse second half, interleave. Pattern: Three-phase (find/reverse/merge).

F8. Copy List with Random Pointer — Medium

Deep copy a list where each node has a random pointer. Hint: Hashmap old→new node. Or interleave clone nodes then split. Pattern: Hashmap-assisted clone.

F9. Add Two Numbers — Medium

Numbers stored as reversed lists; return sum as a list. Example: `2→4→3` + `5→6→4` = `342` + `465` = `807` → `7→0→8`. Hint: Walk both with a carry. Pattern: Simulation with carry.

F10. Palindrome Linked List — Easy

Is the list a palindrome? Hint: Find middle, reverse second half, compare. Pattern: Find middle + reverse.

F11. Intersection of Two Linked Lists — Easy

Find the node where two lists intersect. Hint: Two pointers that switch heads when they hit null — they meet at the intersection (or both hit null together). Pattern: Two pointers with switch.

F12. LRU Cache — Medium

(Duplicate of C15, revisited.)

Category G — Binary Search (10 problems)

G1. Binary Search — Easy

Basic search in a sorted array. Pattern: Standard binary search.

G2. First Bad Version — Easy

Binary search for the first bad version given `isBadVersion(k)`. Pattern: First-true binary search.

G3. Search Insert Position — Easy

Find target index, or where it would be inserted. Hint: `bisect_left`. Pattern: Binary search.

G4. Find Peak Element — Medium

Return any peak in the array (neighbors both smaller). Hint: Binary search — go toward the larger neighbor. Pattern: Binary search on non-sorted.

G5. Search in Rotated Sorted Array — Medium

(Duplicate of A8.)

G6. Find Min in Rotated Sorted Array — Medium

(Duplicate of A7.)

G7. Koko Eating Bananas — Medium

Min eating speed to finish all piles in h hours. Example: `piles=[3,6,7,11]`, $h=8 \rightarrow 4$ Hint: Binary search on the answer (speed). Pattern: Binary search on answer.

G8. Capacity To Ship Packages Within D Days — Medium

Minimum ship capacity to ship all packages in D days. Hint: Binary search on capacity; check feasibility linearly. Pattern: Binary search on answer.

G9. Median of Two Sorted Arrays — Hard

(Duplicate of A20.)

G10. Time Based Key-Value Store — Medium

`set(key, value, timestamp)`, `get(key, timestamp)` returning value at largest $ts \leq timestamp$. Hint: Hashmap of key $\rightarrow$ sorted list of (timestamp, value); binary search on lookup. Pattern: Hashmap + binary search.

Category H — Recursion & Backtracking (12 problems)

H1. Factorial / Power — Easy

Compute $n!$ or x^n recursively. Hint: Base cases matter. Power in $O(\log n)$ via halving. Pattern: Basic recursion.

H2. Fibonacci — Easy

Return n th Fibonacci. Hint: Memoize or iterate — never plain recursion for large n . Pattern: Memoization.

H3. Climbing Stairs — Easy

How many ways to climb n stairs with 1 or 2 steps at a time? Hint: $f(n) = f(n-1) + f(n-2)$ — that's Fibonacci. Pattern: DP /

recursion.

H4. Subsets — Medium

All subsets of a set (power set). Example: `[1,2,3] → [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]` Hint: At each element, either include or exclude. Backtracking or bit manipulation. Pattern: Backtracking / enumeration.

H5. Subsets II — Medium

Same but input has duplicates; return unique subsets only. Hint: Sort first. Skip duplicates at same recursion depth. Pattern: Backtracking with dedup.

H6. Permutations — Medium

All permutations of distinct integers. Hint: Backtracking; swap or "used" set. Pattern: Backtracking.

H7. Permutations II — Medium

Input has duplicates; return unique permutations. Hint: Sort. Skip a duplicate if its predecessor wasn't used at this level. Pattern: Backtracking with dedup.

H8. Combination Sum — Medium

Find all unique combos summing to target (elements can be reused). Hint: Backtracking; pass current index to avoid duplicate combos. Pattern: Backtracking with pruning.

H9. Word Search — Medium

Does a 2D board contain the word (adjacent, no reuse)? Hint: DFS from each cell; mark visited, unmark on backtrack. Pattern: Grid DFS + backtracking.

H10. N-Queens — Hard

Place N queens on an NxN board with none attacking. Hint: Row-by-row backtracking; track columns, diagonals in sets. Pattern: Constraint backtracking.

H11. Sudoku Solver — Hard

Fill in a valid Sudoku. Hint: For each empty cell, try 1-9; recurse; undo on failure. Pattern: Constraint backtracking.

H12. Letter Combinations of Phone Number — Medium

Given digits like "23", return all letter combinations. Hint: Backtracking through mapping. Pattern: Backtracking / DFS.

Category I — Trees & Binary Search Trees (20 problems)

I1. Maximum Depth of Binary Tree — Easy

Hint: `1 + max(depth(left), depth(right))`. Base: null → 0. Pattern: Simple recursion.

I2. Same Tree — Easy

Are two trees identical? Hint: Recurse both together; both null → true; one null → false; values match → recurse. Pattern: Parallel recursion.

I3. Invert Binary Tree — Easy

Mirror the tree. Hint: Swap children, recurse. Pattern: Recursion.

I4. Symmetric Tree — Easy

Is the tree symmetric around its center? Hint: Recurse on (left.left vs right.right) and (left.right vs right.left). Pattern: Parallel recursion.

15. Path Sum — Easy

Does any root-to-leaf path sum to target? Hint: Subtract node.val from target as you recurse; check leaf. Pattern: DFS with running total.

16. Binary Tree Level Order Traversal — Medium

Return values grouped by level. Hint: BFS with queue; capture level size at each iteration. Pattern: BFS.

17. Binary Tree Zigzag Level Order — Medium

Level-order but alternating direction each level. Hint: BFS; reverse each odd level. Pattern: BFS.

18. Binary Tree Right Side View — Medium

Values seen from the right side. Hint: BFS taking last node per level, or DFS visiting right first. Pattern: BFS / DFS.

19. Lowest Common Ancestor of BST — Easy

Hint: Descend left if both p,q < root; right if both >; else root is LCA. Pattern: BST property traversal.

110. Lowest Common Ancestor of Binary Tree — Medium

Hint: Recurse both subtrees; if both return non-null, current node is LCA. Pattern: Post-order recursion.

111. Validate BST — Medium

Is the tree a valid BST? Hint: In-order traversal is strictly increasing. Or pass (min, max) bounds recursively. Pattern: In-order / bounds recursion.

112. Kth Smallest Element in BST — Medium

Hint: In-order traversal, return kth visited. Pattern: In-order.

113. Serialize and Deserialize Binary Tree — Hard

Convert tree to string and back. Hint: Pre-order with null markers. Deserialize via queue. Pattern: Traversal-based serialization.

114. Construct Binary Tree from Preorder and Inorder — Medium

Hint: First of preorder is root. Split inorder at root to get left/right subtrees. Pattern: Divide-and-conquer.

115. Diameter of Binary Tree — Easy

Longest path (in edges) between any two nodes. Hint: For each node, diameter through it = left_depth + right_depth. Track global max. Pattern: Post-order recursion with global state.

116. Balanced Binary Tree — Easy

Is height difference at every node ≤ 1 ? Hint: Post-order; return -1 to signal unbalanced. Pattern: Post-order recursion.

117. Binary Tree Maximum Path Sum — Hard

Max sum of any path (may not go through root). Hint: At each node, `gain = max(0, gain(child))`. Path through node = node + left_gain + right_gain. Pattern: Post-order with global max.

118. Count Good Nodes in Binary Tree — Medium

Node is "good" if no ancestor has value > it. Hint: DFS carrying max-so-far. Pattern: DFS with state.

119. Sum Root to Leaf Numbers — Medium

Each root-to-leaf path represents a number. Sum them. Hint: DFS; carry current number, multiply by 10 as you descend. Pattern: DFS with accumulator.

I20. Flatten Binary Tree to Linked List — Medium

Flatten to right-only "linked list" in pre-order. Hint: Reverse post-order (right, left, root); track previous processed node. Pattern: Reverse pre-order traversal.

Category J — Heaps / Priority Queues (8 problems)

J1. Kth Largest Element in Array — Medium

Hint: Min-heap of size k. Or quickselect for $O(n)$ average. Pattern: Heap / quickselect.

J2. Kth Largest Element in Stream — Easy

Design a class that supports `add(val)` returning kth largest so far. Hint: Maintain a min-heap of size k. Pattern: Heap.

J3. Top K Frequent Elements — Medium

(Duplicate of C7.)

J4. Find Median from Data Stream — Hard

Class supporting `addNum` and `findMedian`. Hint: Two heaps — max-heap for lower half, min-heap for upper. Rebalance. Pattern: Two heaps.

J5. Merge K Sorted Lists — Hard

(Duplicate of F5.)

J6. K Closest Points to Origin — Medium

K closest points to (0,0). Hint: Heap of size k with negated distance, or quickselect. Pattern: Heap.

J7. Task Scheduler — Medium

Given tasks and cooldown n, min intervals to finish all. Hint: Max-heap of frequencies; simulate with cooldown queue. Or formula-based. Pattern: Greedy with heap.

J8. Reorganize String — Medium

Rearrange so no two adjacent chars are the same. Hint: Max-heap by count; always take top two. Pattern: Greedy with heap.

Category K — Graphs (15 problems)

K1. Number of Islands — Medium

Count connected components of '1's in a 2D grid. Hint: DFS/BFS from each unvisited '1', marking visited. Pattern: Grid DFS/BFS.

K2. Max Area of Island — Medium

Largest area of connected '1's. Hint: DFS returning size. Pattern: Grid DFS.

K3. Flood Fill — Easy

Paint-bucket fill. Hint: DFS/BFS from starting cell. Pattern: Grid DFS.

K4. Rotting Oranges — Medium

Min minutes until all fresh oranges rot; -1 if impossible. Hint: Multi-source BFS from all initially rotten. Pattern: Multi-source BFS.

K5. Walls and Gates — Medium

Fill each empty room with distance to nearest gate. Hint: Multi-source BFS from all gates. Pattern: Multi-source BFS.

K6. Course Schedule — Medium

Can you finish all courses given prerequisites (detect cycle in directed graph)? Hint: Topological sort via Kahn's (BFS on in-degrees) or DFS with 3-coloring. Pattern: Topological sort.

K7. Course Schedule II — Medium

Return a valid course order. Hint: Same as K6, output the order. Pattern: Topological sort.

K8. Clone Graph — Medium

Deep clone an undirected graph. Hint: BFS/DFS with hashmap `original → clone`. Pattern: Graph traversal + hashmap.

K9. Pacific Atlantic Water Flow — Medium

Cells where water flows to both oceans. Hint: DFS from both oceans inward; intersect reachable sets. Pattern: Multi-source DFS.

K10. Word Ladder — Hard

Shortest transformation from beginWord to endWord, changing one letter at a time (each intermediate must be in the dictionary). Hint: BFS. Precompute "ord", "wrđ" pattern buckets to find neighbors fast. Pattern: BFS + pattern hashing.

K11. Number of Connected Components in Graph — Medium

Given n nodes and edges, count components. Hint: Union-Find or DFS. Pattern: Union-Find / DFS.

K12. Graph Valid Tree — Medium

Do n nodes + edges form a tree (connected, no cycles)? Hint: Union-Find (any union fails → cycle) or edges = n-1 and connected. Pattern: Union-Find.

K13. Dijkstra's Shortest Path — Medium

Shortest path from source to all nodes in weighted graph (no negatives). Hint: Min-heap of (distance, node); relax neighbors. Pattern: Dijkstra.

K14. Network Delay Time — Medium

Time for signal to reach all nodes from source. Hint: Dijkstra; answer is the max distance (∞ if unreachable). Pattern: Dijkstra.

K15. Alien Dictionary — Hard

Given sorted alien-language words, deduce letter order. Hint: Compare adjacent word pairs to build graph edges; topological sort. Pattern: Graph construction + topo sort.

Category L — Dynamic Programming (15 problems)

L1. Climbing Stairs — Easy

(Duplicate of H3.)

L2. House Robber — Medium

Max money robbable from a row of houses (can't rob adjacent). Hint: `dp[i] = max(dp[i-1], dp[i-2] + nums[i])`. Pattern: Linear DP.

L3. House Robber II — Medium

Same but houses are in a circle. Hint: Two passes — exclude first or exclude last. Pattern: Linear DP variation.

L4. Coin Change — Medium

Min coins to make amount, or -1. Hint: $dp[i] = \min(dp[i - \text{coin}] + 1)$ for each coin. Pattern: Unbounded knapsack.

L5. Coin Change II — Medium

Number of ways to make amount (order doesn't matter). Hint: Iterate coins outer, amount inner. Pattern: Unbounded knapsack (count).

L6. Longest Increasing Subsequence — Medium

Length of longest strictly increasing subsequence. Hint: $O(n^2)$ DP or $O(n \log n)$ with binary search + patience piles. Pattern: LIS.

L7. Longest Common Subsequence — Medium

LCS of two strings. Hint: 2D DP; if chars match, $dp[i][j] = dp[i-1][j-1] + 1$. Pattern: 2D DP.

L8. Edit Distance — Medium

Min edits (insert/delete/replace) to convert word1 → word2. Hint: 2D DP; cases match / insert / delete / replace. Pattern: 2D DP (Levenshtein).

L9. Unique Paths — Medium

Robot at top-left, target bottom-right, moves right/down only. Hint: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. Or math: $C(m+n-2, m-1)$. Pattern: Grid DP.

L10. Minimum Path Sum — Medium

Min sum path in a grid, moving right/down. Hint: $dp[i][j] = \text{grid}[i][j] + \min(dp[i-1][j], dp[i][j-1])$. Pattern: Grid DP.

L11. Word Break — Medium

(Duplicate of B15.)

L12. Partition Equal Subset Sum — Medium

Can array be split into two subsets with equal sum? Hint: Subset-sum DP for target = total/2. Pattern: 0/1 knapsack.

L13. Best Time to Buy and Sell Stock with Cooldown — Medium

After selling, must skip a day before buying. Hint: Three states — holding, sold, resting. Pattern: State-machine DP.

L14. Decode Ways — Medium

Number of ways to decode "12" (AB / L) etc. Hint: $dp[i] = (dp[i-1] \text{ if } s[i-1] \neq '0') + (dp[i-2] \text{ if } s[i-2:i] \text{ in } 10..26)$. Pattern: 1D DP.

L15. Longest Palindromic Subsequence — Medium

Longest palindromic subseq (not substring). Hint: 2D DP; equivalent to LCS of s and reversed s. Pattern: 2D DP.

Category M — Greedy, Bit Manipulation, Design (14 problems)

M1. Jump Game — Medium

Can you reach the last index (each element is max jump)? Hint: Track farthest reachable so far; if current index > farthest, fail.

Pattern: Greedy.

M2. Jump Game II — Medium

Min jumps to reach end. Hint: BFS-style greedy — at each "level" jump to the farthest reachable. Pattern: Greedy.

M3. Gas Station — Medium

Circular route: starting from which station can you complete a lap? Hint: If total gas $\geq$ total cost, answer exists. Reset start when running total goes negative. Pattern: Greedy.

M4. Partition Labels — Medium

Partition string so each letter appears in at most one part; maximize parts. Hint: Record last index of each char; extend partition to cover all chars in current range. Pattern: Greedy with lookup.

M5. Meeting Rooms II — Medium

Min meeting rooms needed. Hint: Sort starts and ends separately; two pointers count overlaps. Or heap of end times. Pattern: Greedy / heap.

M6. Single Number — Easy

Every element appears twice except one; find it. Hint: XOR all — pairs cancel. Pattern: XOR.

M7. Number of 1 Bits — Easy

Count set bits. Hint: `x & (x-1)` clears the lowest set bit; count iterations. Pattern: Bit trick.

M8. Counting Bits — Easy

For $0..n$, return array of bit counts. Hint: `dp[i] = dp[i >> 1] + (i & 1)`. Pattern: Bit DP.

M9. Missing Number — Easy

$0..n$ with one missing; find it. Hint: XOR of range vs XOR of array. Or sum formula. Pattern: XOR / math.

M10. Reverse Bits — Easy

Reverse bits of a 32-bit integer. Hint: Shift result left, OR with lowest bit of n , shift n right. Pattern: Bit manipulation.

M11. Sum of Two Integers (No + or -) — Medium

Hint: XOR gives sum without carry; AND-shift gives carry. Loop until carry = 0. Pattern: Bit arithmetic.

M12. Design Twitter — Medium

Design `postTweet`, `getNewsFeed` (10 latest from followees + self), `follow`, `unfollow`. Hint: Hashmap of user→tweets; hashmap of user→followees. Merge k sorted with heap for feed. Pattern: OOP + heap merge.

M13. Design Tic-Tac-Toe — Medium

Detect winner in $O(1)$ per move. Hint: Track row/col/diag counts. Pattern: Amortized state tracking.

M14. Design Snake Game — Medium

Hint: Deque for snake body; set for $O(1)$ collision check. Pattern: Deque + set.

That's ~180 problems across 13 categories. Solutions follow in **Part IX**.

PART IX — SOLUTIONS

Read only after you've written code. Each solution includes: approach, code, complexity, and the key insight.

Solutions — Category A (Arrays & Lists)

A1. Two Sum

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
```

Complexity: $O(n)$ / $O(n)$. **Insight:** Every "find pair with property" problem should trigger "can a hashmap eliminate the inner loop?"

A2. Best Time to Buy and Sell Stock

```
def max_profit(prices):
    min_price, best = float('inf'), 0
    for p in prices:
        min_price = min(min_price, p)
        best = max(best, p - min_price)
    return best
```

Complexity: $O(n)$ / $O(1)$. **Insight:** For each day, ask "what's the cheapest day so far?" — one pass suffices.

A3. Contains Duplicate

```
def contains_duplicate(nums):
    return len(nums) != len(set(nums))
```

Complexity: $O(n)$ / $O(n)$.

A4. Maximum Subarray (Kadane's)

```
def max_subarray(nums):
    best = current = nums[0]
    for n in nums[1:]:
        current = max(n, current + n)
        best = max(best, current)
    return best
```

Complexity: $O(n)$ / $O(1)$. **Insight:** At each element, decide: extend the run or restart. This "local vs global" pattern is the seed of DP.

A5. Product of Array Except Self

```
def product_except_self(nums):
    n = len(nums)
    output = [1] * n
    left = 1
    for i in range(n):
        output[i] = left
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):
        output[i] *= right
        right *= nums[i]
    return output
```

Complexity: $O(n)$ / $O(1)$ extra. **Insight:** Two sweeps — left-products then right-products — done in-place.

A6. Maximum Product Subarray

```
def max_product(nums):
    max_p = min_p = best = nums[0]
    for n in nums[1:]:
        if n < 0:
            max_p, min_p = min_p, max_p
            max_p = max(n, max_p * n)
            min_p = min(n, min_p * n)
            best = max(best, max_p)
    return best
```

Complexity: $O(n)$ / $O(1)$. **Insight:** Track both extremes; negatives flip them.

A7. Find Minimum in Rotated Sorted Array

```
def find_min(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]:
            lo = mid + 1
        else:
            hi = mid
    return nums[lo]
```

Complexity: $O(\log n)$. **Insight:** Compare mid to hi to know which half is sorted.

A8. Search in Rotated Sorted Array

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        if nums[lo] <= nums[mid]: # left half sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else: # right half sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Complexity: $O(\log n)$. **Insight:** One half is always sorted; test if target is in it.

A9. Merge Intervals

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

Complexity: $O(n \log n)$ / $O(n)$.

A10. Insert Interval

```
def insert(intervals, new):
    result, i, n = [], 0, len(intervals)
    while i < n and intervals[i][1] < new[0]:
        result.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= new[1]:
        new = [min(new[0], intervals[i][0]), max(new[1], intervals[i][1])]
        i += 1
    result.append(new)
    result.extend(intervals[i:])
    return result
```

Complexity: $O(n)$.

A11. Non-Overlapping Intervals

```
def erase_overlap(intervals):
    intervals.sort(key=lambda x: x[1])
    end, removed = float('-inf'), 0
    for s, e in intervals:
        if s >= end:
            end = e
        else:
            removed += 1
    return removed
```

Complexity: $O(n \log n)$. **Insight:** Greedy by earliest end.

A12. Rotate Array

```
def rotate(nums, k):
    k %= len(nums)
    nums.reverse()
    nums[:k] = reversed(nums[:k])
    nums[k:] = reversed(nums[k:])
```

Complexity: $O(n) / O(1)$.

A13. Move Zeroes

```
def move_zeroes(nums):
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1
```

Complexity: $O(n) / O(1)$.

A14. Container With Most Water

```
def max_area(height):
    lo, hi, best = 0, len(height) - 1, 0
    while lo < hi:
        best = max(best, (hi - lo) * min(height[lo], height[hi]))
        if height[lo] < height[hi]:
            lo += 1
        else:
            hi -= 1
    return best
```

Complexity: $O(n)$. **Insight:** Moving the taller side can never help — width shrinks and height is bounded by the shorter side.

A15. 3Sum

```
def three_sum(nums):
    nums.sort()
    result = []
    for i in range(len(nums) - 2):
        if i and nums[i] == nums[i-1]: continue
        lo, hi = i + 1, len(nums) - 1
        while lo < hi:
            s = nums[i] + nums[lo] + nums[hi]
            if s < 0: lo += 1
            elif s > 0: hi -= 1
            else:
                result.append([nums[i], nums[lo], nums[hi]])
                while lo < hi and nums[lo] == nums[lo+1]: lo += 1
                while lo < hi and nums[hi] == nums[hi-1]: hi -= 1
                lo += 1; hi -= 1
    return result
```

Complexity: $O(n^2)$.

A16. Trapping Rain Water

```
def trap(height):
    lo, hi = 0, len(height) - 1
    left_max = right_max = trapped = 0
    while lo < hi:
        if height[lo] < height[hi]:
            left_max = max(left_max, height[lo])
            trapped += left_max - height[lo]
            lo += 1
        else:
            right_max = max(right_max, height[hi])
            trapped += right_max - height[hi]
            hi -= 1
    return trapped
```

Complexity: $O(n)$ / $O(1)$. **Insight:** Whichever side is shorter is the binding constraint at that step.

A17. Subarray Sum Equals K

```
def subarray_sum(nums, k):
    from collections import defaultdict
    prefix_counts = defaultdict(int); prefix_counts[0] = 1
    running = count = 0
    for n in nums:
        running += n
        count += prefix_counts[running - k]
        prefix_counts[running] += 1
    return count
```

Complexity: $O(n)$. **Insight:** `running - k` seen before means a subarray from there to here sums to k.

A18. Find All Duplicates

```
def find_duplicates(nums):
    result = []
    for x in nums:
        idx = abs(x) - 1
        if nums[idx] < 0:
            result.append(idx + 1)
        else:
            nums[idx] = -nums[idx]
    return result
```

Complexity: $O(n)$ / $O(1)$.

A19. Next Permutation

```
def next_permutation(nums):
    i = len(nums) - 2
    while i >= 0 and nums[i] >= nums[i+1]: i -= 1
    if i >= 0:
        j = len(nums) - 1
        while nums[j] <= nums[i]: j -= 1
        nums[i], nums[j] = nums[j], nums[i]
    nums[i+1:] = reversed(nums[i+1:])
```

Complexity: $O(n)$.

A20. Median of Two Sorted Arrays

```
def find_median(a, b):
    if len(a) > len(b): a, b = b, a
    m, n = len(a), len(b)
    lo, hi = 0, m
    while lo <= hi:
        i = (lo + hi) // 2
        j = (m + n + 1) // 2 - i
        left_a = a[i-1] if i > 0 else float('-inf')
        right_a = a[i] if i < m else float('inf')
        left_b = b[j-1] if j > 0 else float('-inf')
        right_b = b[j] if j < n else float('inf')
        if left_a <= right_b and left_b <= right_a:
            if (m + n) % 2:
                return max(left_a, left_b)
            return (max(left_a, left_b) + min(right_a, right_b)) / 2
        elif left_a > right_b:
            hi = i - 1
        else:
            lo = i + 1
```

Complexity: $O(\log(\min(m,n)))$.

Solutions — Category B (Strings)

B1. Valid Palindrome

```
def is_palindrome(s):
    lo, hi = 0, len(s) - 1
    while lo < hi:
        while lo < hi and not s[lo].isalnum(): lo += 1
        while lo < hi and not s[hi].isalnum(): hi -= 1
        if s[lo].lower() != s[hi].lower(): return False
        lo += 1; hi -= 1
    return True
```

B2. Valid Anagram

```
from collections import Counter
def is_anagram(s, t):
    return Counter(s) == Counter(t)
```

B3. Longest Common Prefix

```
def longest_common_prefix(strs):
    if not strs: return ""
    prefix = ""
    for chars in zip(*strs):
        if len(set(chars)) == 1:
            prefix += chars[0]
        else:
            break
    return prefix
```

B4. Reverse Words

```
def reverse_words(s):
    return " ".join(s.split()[::-1])
```

B5. Longest Substring Without Repeating Characters

```
def length_of_longest_substring(s):
    seen, left, best = set(), 0, 0
    for right, ch in enumerate(s):
        while ch in seen:
            seen.remove(s[left]); left += 1
        seen.add(ch)
        best = max(best, right - left + 1)
    return best
```

Complexity: $O(n)$.

B6. Longest Repeating Character Replacement

```
def character_replacement(s, k):
    from collections import Counter
    counts, left, best, max_freq = Counter(), 0, 0, 0
    for right, ch in enumerate(s):
        counts[ch] += 1
        max_freq = max(max_freq, counts[ch])
        if right - left + 1 - max_freq > k:
            counts[s[left]] -= 1; left += 1
        best = max(best, right - left + 1)
    return best
```

B7. Minimum Window Substring

```
def min_window(s, t):
    from collections import Counter
    if not t: return ""
    need = Counter(t)
    missing = len(t)
    left = start = 0; end = float('inf')
    for right, ch in enumerate(s, 1):
        if need[ch] > 0: missing -= 1
        need[ch] -= 1
        if missing == 0:
            while need[s[left]] < 0:
                need[s[left]] += 1; left += 1
            if right - left < end - start:
                start, end = left, right
            need[s[left]] += 1; missing += 1; left += 1
    return "" if end == float('inf') else s[start:end]
```

B8. Group Anagrams

```
from collections import defaultdict
def group_anagrams(strs):
    groups = defaultdict(list)
    for w in strs:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())
```

B9. Valid Parentheses

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

B10. Generate Parentheses

```
def generate_parenthesis(n):
    result = []
    def backtrack(cur, open_c, close_c):
        if len(cur) == 2 * n:
            result.append(cur); return
        if open_c < n: backtrack(cur + "(", open_c + 1, close_c)
        if close_c < open_c: backtrack(cur + ")", open_c, close_c + 1)
    backtrack("", 0, 0)
    return result
```

B11. String to Integer

```
def my_atoi(s):
    s = s.lstrip()
    if not s: return 0
    sign = 1; i = 0
    if s[0] in "+-":
        sign = -1 if s[0] == '-' else 1; i = 1
    num = 0
    while i < len(s) and s[i].isdigit():
        num = num * 10 + int(s[i]); i += 1
    num *= sign
    return max(-2**31, min(2**31 - 1, num))
```

B12. Longest Palindromic Substring

```
def longest_palindrome(s):
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1; r += 1
        return s[l+1:r]
    best = ""
    for i in range(len(s)):
        for candidate in (expand(i, i), expand(i, i+1)):
            if len(candidate) > len(best): best = candidate
    return best
```

B13. Palindromic Substrings

```
def count_substrings(s):
    def expand(l, r):
        c = 0
        while l >= 0 and r < len(s) and s[l] == s[r]:
            c += 1; l -= 1; r += 1
        return c
    return sum(expand(i, i) + expand(i, i+1) for i in range(len(s)))
```

B14. Encode and Decode Strings

```
class Codec:
    def encode(self, strs):
        return "".join(f"{len(s)}#{s}" for s in strs)
    def decode(self, s):
        result, i = [], 0
        while i < len(s):
            j = s.index('#', i)
            length = int(s[i:j])
            result.append(s[j+1:j+1+length])
            i = j + 1 + length
        return result
```

B15. Word Break

```
def word_break(s, word_dict):
    word_set = set(word_dict)
    dp = [False] * (len(s) + 1); dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True; break
    return dp[-1]
```

Solutions — Category C (Hashmaps & Sets)

C1. First Unique Character

```
from collections import Counter
def first_uniq_char(s):
    counts = Counter(s)
    for i, ch in enumerate(s):
        if counts[ch] == 1: return i
    return -1
```

C2. Ransom Note

```
from collections import Counter
def can_construct(note, magazine):
    return not (Counter(note) - Counter(magazine))
```

C3. Isomorphic Strings

```
def is_isomorphic(s, t):
    return len(set(zip(s, t))) == len(set(s)) == len(set(t))
```

C4. Word Pattern

```
def word_pattern(pattern, s):
    words = s.split()
    if len(pattern) != len(words): return False
    return len(set(zip(pattern, words))) == len(set(pattern)) == len(set(words))
```

C5. Happy Number

```
def is_happy(n):
    seen = set()
    while n != 1 and n not in seen:
        seen.add(n)
        n = sum(int(d)**2 for d in str(n))
    return n == 1
```

C6. Longest Consecutive Sequence

```
def longest_consecutive(nums):
    num_set = set(nums)
    best = 0
    for n in num_set:
        if n - 1 not in num_set:
            length = 1
            while n + length in num_set: length += 1
            best = max(best, length)
    return best
```

C7. Top K Frequent Elements

```
from collections import Counter
def top_k_frequent(nums, k):
    return [x for x, _ in Counter(nums).most_common(k)]
```

C8. Subarray Sum Equals K

(Same as A17.)

C9. Longest Substring with At Most K Distinct

```
from collections import defaultdict
def length_of_longest_k_distinct(s, k):
    counts = defaultdict(int); left = best = 0
    for right, ch in enumerate(s):
        counts[ch] += 1
        while len(counts) > k:
            counts[s[left]] -= 1
            if counts[s[left]] == 0: del counts[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best
```

C10. Two Sum II (Sorted)

```
def two_sum_ii(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        s = nums[lo] + nums[hi]
        if s == target: return [lo+1, hi+1]
        elif s < target: lo += 1
        else: hi -= 1
```

C11. Group Shifted Strings

```
from collections import defaultdict
def group_strings(strs):
    groups = defaultdict(list)
    for s in strs:
        key = tuple((ord(c) - ord(s[0])) % 26 for c in s)
        groups[key].append(s)
    return list(groups.values())
```

C12. Design HashMap

```

class MyHashMap:
    def __init__(self):
        self.size = 1000
        self.buckets = [[] for _ in range(self.size)]
    def _bucket(self, key): return self.buckets[key % self.size]
    def put(self, key, value):
        b = self._bucket(key)
        for i, (k, _) in enumerate(b):
            if k == key: b[i] = (key, value); return
        b.append((key, value))
    def get(self, key):
        for k, v in self._bucket(key):
            if k == key: return v
        return -1
    def remove(self, key):
        b = self._bucket(key)
        for i, (k, _) in enumerate(b):
            if k == key: b.pop(i); return

```

C13. Random Pick with Weight

```

import random, bisect
class Solution:
    def __init__(self, w):
        self.prefix = []
        total = 0
        for x in w:
            total += x
            self.prefix.append(total)
        self.total = total
    def pickIndex(self):
        return bisect.bisect_left(self.prefix, random.randint(1, self.total))

```

C14. Insert Delete GetRandom O(1)

```

import random
class RandomizedSet:
    def __init__(self):
        self.items = []; self.idx = {}
    def insert(self, val):
        if val in self.idx: return False
        self.idx[val] = len(self.items); self.items.append(val)
        return True
    def remove(self, val):
        if val not in self.idx: return False
        i = self.idx[val]; last = self.items[-1]
        self.items[i] = last; self.idx[last] = i
        self.items.pop(); del self.idx[val]
        return True
    def getRandom(self):
        return random.choice(self.items)

```

C15. LRU Cache

```

from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity):
        self.cache = OrderedDict(); self.capacity = capacity
    def get(self, key):
        if key not in self.cache: return -1
        self.cache.move_to_end(key)
        return self.cache[key]
    def put(self, key, value):
        if key in self.cache: self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)

```

Solutions — Category D (Two Pointers & Sliding Window)

D1. Reverse String

```

def reverse_string(s):
    lo, hi = 0, len(s) - 1
    while lo < hi:
        s[lo], s[hi] = s[hi], s[lo]; lo += 1; hi -= 1

```

D2. Squares of Sorted Array

```
def sorted_squares(nums):
    n = len(nums); result = [0]*n
    lo, hi, i = 0, n-1, n-1
    while lo <= hi:
        if abs(nums[lo]) > abs(nums[hi]):
            result[i] = nums[lo]**2; lo += 1
        else:
            result[i] = nums[hi]**2; hi -= 1
        i -= 1
    return result
```

D3. Remove Duplicates from Sorted Array

```
def remove_duplicates(nums):
    if not nums: return 0
    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[read-1]:
            nums[write] = nums[read]; write += 1
    return write
```

D4. Remove Element

```
def remove_element(nums, val):
    write = 0
    for read in range(len(nums)):
        if nums[read] != val:
            nums[write] = nums[read]; write += 1
    return write
```

D5. Sort Colors

```
def sort_colors(nums):
    lo, mid, hi = 0, 0, len(nums) - 1
    while mid <= hi:
        if nums[mid] == 0:
            nums[lo], nums[mid] = nums[mid], nums[lo]; lo += 1; mid += 1
        elif nums[mid] == 2:
            nums[mid], nums[hi] = nums[hi], nums[mid]; hi -= 1
        else:
            mid += 1
```

D6. Maximum Average Subarray

```
def find_max_average(nums, k):
    window = sum(nums[:k]); best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i-k]
        best = max(best, window)
    return best / k
```

D7. Same as B6.

D8. Fruit Into Baskets

```
from collections import defaultdict
def total_fruit(fruits):
    counts = defaultdict(int); left = best = 0
    for right, f in enumerate(fruits):
        counts[f] += 1
        while len(counts) > 2:
            counts[fruits[left]] -= 1
            if counts[fruits[left]] == 0: del counts[fruits[left]]
            left += 1
        best = max(best, right - left + 1)
    return best
```

D9. Permutation in String

```

from collections import Counter
def check_inclusion(s1, s2):
    if len(s1) > len(s2): return False
    need, window = Counter(s1), Counter(s2[:len(s1)])
    if need == window: return True
    for i in range(len(s1), len(s2)):
        window[s2[i]] += 1
        window[s2[i-len(s1)]] -= 1
        if window[s2[i-len(s1)]] == 0: del window[s2[i-len(s1)]]
        if need == window: return True
    return False

```

D10. Find All Anagrams

```

from collections import Counter
def find_anagrams(s, p):
    if len(p) > len(s): return []
    need, window, result = Counter(p), Counter(s[:len(p)]), []
    if need == window: result.append(0)
    for i in range(len(p), len(s)):
        window[s[i]] += 1
        window[s[i-len(p)]] -= 1
        if window[s[i-len(p)]] == 0: del window[s[i-len(p)]]
        if need == window: result.append(i - len(p) + 1)
    return result

```

D11. Subarrays with Product Less Than K

```

def num_subarray_product_less_than_k(nums, k):
    if k <= 1: return 0
    prod, left, count = 1, 0, 0
    for right, n in enumerate(nums):
        prod *= n
        while prod >= k:
            prod //= nums[left]; left += 1
        count += right - left + 1
    return count

```

D12. Minimum Size Subarray Sum

```

def min_subarray_len(target, nums):
    left = 0; total = 0; best = float('inf')
    for right, n in enumerate(nums):
        total += n
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]; left += 1
    return 0 if best == float('inf') else best

```

Solutions — Category E (Stacks & Queues)

E1. Same as B9.

E2. Min Stack

```

class MinStack:
    def __init__(self):
        self.stack = []; self.mins = []
    def push(self, val):
        self.stack.append(val)
        self.mins.append(min(val, self.mins[-1]) if self.mins else val)
    def pop(self): self.stack.pop(); self.mins.pop()
    def top(self): return self.stack[-1]
    def getMin(self): return self.mins[-1]

```

E3. Evaluate Reverse Polish Notation

```
def eval_rpn(tokens):
    stack = []
    for t in tokens:
        if t in "+-*/*":
            b, a = stack.pop(), stack.pop()
            stack.append({"+": a+b, "-": a-b, "*": a*b, "/": int(a/b)}[t])
        else:
            stack.append(int(t))
    return stack[0]
```

E4. Daily Temperatures

```
def daily_temperatures(temps):
    result = [0]*len(temps); stack = []
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop(); result[j] = i - j
        stack.append(i)
    return result
```

E5. Next Greater Element

```
def next_greater(nums):
    result = [-1]*len(nums); stack = []
    for i, n in enumerate(nums):
        while stack and nums[stack[-1]] < n:
            result[stack.pop()] = n
        stack.append(i)
    return result
```

E6. Largest Rectangle in Histogram

```
def largest_rectangle_area(heights):
    stack = []; best = 0
    heights.append(0)
    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] > h:
            top = stack.pop()
            width = i if not stack else i - stack[-1] - 1
            best = max(best, heights[top] * width)
        stack.append(i)
    heights.pop()
    return best
```

E7. Sliding Window Maximum

```
from collections import deque
def max_sliding_window(nums, k):
    dq, result = deque(), []
    for i, n in enumerate(nums):
        while dq and nums[dq[-1]] < n: dq.pop()
        dq.append(i)
        if dq[0] <= i - k: dq.popleft()
        if i >= k - 1: result.append(nums[dq[0]])
    return result
```

E8. Queue Using Stacks

```
class MyQueue:
    def __init__(self):
        self.inn, self.out = [], []
    def push(self, x): self.inn.append(x)
    def _shift(self):
        if not self.out:
            while self.inn: self.out.append(self.inn.pop())
    def pop(self): self._shift(); return self.out.pop()
    def peek(self): self._shift(); return self.out[-1]
    def empty(self): return not (self.inn or self.out)
```

E9. Stack Using Queues

```

from collections import deque
class MyStack:
    def __init__(self): self.q = deque()
    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q) - 1): self.q.append(self.q.popleft())
    def pop(self): return self.q.popleft()
    def top(self): return self.q[0]
    def empty(self): return not self.q

```

E10. Decode String

```

def decode_string(s):
    stack = []; cur = ""; k = 0
    for ch in s:
        if ch.isdigit(): k = k*10 + int(ch)
        elif ch == '[':
            stack.append((cur, k)); cur = ""; k = 0
        elif ch == ']':
            prev, times = stack.pop()
            cur = prev + cur * times
        else:
            cur += ch
    return cur

```

E11. Asteroid Collision

```

def asteroid_collision(asteroids):
    stack = []
    for a in asteroids:
        while stack and a < 0 < stack[-1]:
            if stack[-1] < -a: stack.pop(); continue
            elif stack[-1] == -a: stack.pop()
            break
        else:
            stack.append(a)
    return stack

```

E12. Basic Calculator

```

def calculate(s):
    stack = []; num = 0; sign = 1; result = 0
    for ch in s:
        if ch.isdigit(): num = num*10 + int(ch)
        elif ch in "+-":
            result += sign * num; num = 0
            sign = 1 if ch == '+' else -1
        elif ch == '(':
            stack.append(result); stack.append(sign)
            result = 0; sign = 1
        elif ch == ')':
            result += sign * num; num = 0
            result *= stack.pop(); result += stack.pop()
    return result + sign * num

```

Solutions — Category F (Linked Lists)

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val; self.next = next

```

F1. Reverse Linked List

```

def reverse_list(head):
    prev, curr = None, head
    while curr:
        curr.next, prev, curr = prev, curr, curr.next
    return prev

```

F2. Detect Cycle

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast: return True
    return False
```

F3. Find Cycle Start

```
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            p = head
            while p is not slow:
                p, slow = p.next, slow.next
            return p
    return None
```

F4. Merge Two Sorted Lists

```
def merge_two_lists(l1, l2):
    dummy = tail = ListNode()
    while l1 and l2:
        if l1.val < l2.val:
            tail.next = l1; l1 = l1.next
        else:
            tail.next = l2; l2 = l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next
```

F5. Merge K Sorted Lists

```
import heapq
def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node: heapq.heappush(heap, (node.val, i, node))
    dummy = tail = ListNode()
    while heap:
        val, i, node = heapq.heappop(heap)
        tail.next = node; tail = node
        if node.next: heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

F6. Remove Nth From End

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head); fast = slow = dummy
    for _ in range(n): fast = fast.next
    while fast.next: fast, slow = fast.next, slow.next
    slow.next = slow.next.next
    return dummy.next
```

F7. Reorder List

```
def reorder_list(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    prev, curr = None, slow.next
    slow.next = None
    while curr:
        curr.next, prev, curr = prev, curr, curr.next
    first, second = head, prev
    while second:
        first.next, second.next, first, second = second, first.next, first.next, second.next
```

F8. Copy List with Random Pointer

```
def copy_random_list(head):
    if not head: return None
    old_to_new = {}
    curr = head
    while curr:
        old_to_new[curr] = Node(curr.val); curr = curr.next
    curr = head
    while curr:
        old_to_new[curr].next = old_to_new.get(curr.next)
        old_to_new[curr].random = old_to_new.get(curr.random)
        curr = curr.next
    return old_to_new[head]
```

F9. Add Two Numbers

```
def add_two_numbers(l1, l2):
    dummy = tail = ListNode(); carry = 0
    while l1 or l2 or carry:
        v = carry + (l1.val if l1 else 0) + (l2.val if l2 else 0)
        carry, v = divmod(v, 10)
        tail.next = ListNode(v); tail = tail.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None
    return dummy.next
```

F10. Palindrome Linked List

```
def is_palindrome(head):
    vals = []
    while head: vals.append(head.val); head = head.next
    return vals == vals[::-1]
```

F11. Intersection of Two Linked Lists

```
def get_intersection_node(a, b):
    p1, p2 = a, b
    while p1 is not p2:
        p1 = b if p1 is None else p1.next
        p2 = a if p2 is None else p2.next
    return p1
```

F12. Same as C15.

Solutions — Category G (Binary Search)

G1. Binary Search

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        elif nums[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

G2. First Bad Version

```
def first_bad_version(n):
    lo, hi = 1, n
    while lo < hi:
        mid = (lo + hi) // 2
        if isBadVersion(mid): hi = mid
        else: lo = mid + 1
    return lo
```

G3. Search Insert Position

```
import bisect
def search_insert(nums, target):
    return bisect.bisect_left(nums, target)
```

G4. Find Peak Element

```
def find_peak_element(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < nums[mid + 1]: lo = mid + 1
        else: hi = mid
    return lo
```

G5/G6/G9. Same as A8, A7, A20.

G7. Koko Eating Bananas

```
import math
def min_eating_speed(piles, h):
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        hours = sum(math.ceil(p / mid) for p in piles)
        if hours <= h: hi = mid
        else: lo = mid + 1
    return lo
```

G8. Ship Packages in D Days

```
def ship_within_days(weights, days):
    def can(cap):
        d = 1; cur = 0
        for w in weights:
            if cur + w > cap: d += 1; cur = 0
            cur += w
        return d <= days
    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if can(mid): hi = mid
        else: lo = mid + 1
    return lo
```

G10. Time Based Key-Value Store

```
from collections import defaultdict
import bisect
class TimeMap:
    def __init__(self): self.store = defaultdict(list)
    def set(self, key, value, timestamp):
        self.store[key].append((timestamp, value))
    def get(self, key, timestamp):
        arr = self.store[key]
        i = bisect.bisect_right(arr, (timestamp, chr(255))) - 1
        return arr[i][1] if i >= 0 else ""
```

Solutions — Category H (Recursion & Backtracking)

H1. Power

```
def my_pow(x, n):
    if n < 0: x, n = 1/x, -n
    result = 1
    while n:
        if n & 1: result *= x
        x *= x; n >>= 1
    return result
```

H2. Fibonacci

```
from functools import lru_cache
@lru_cache(None)
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
```

H3. Climbing Stairs

```
def climb_stairs(n):
    a, b = 1, 1
    for _ in range(n): a, b = b, a + b
    return a
```

H4. Subsets

```
def subsets(nums):
    result = [[]]
    for n in nums:
        result += [curr + [n] for curr in result]
    return result
```

H5. Subsets II

```
def subsets_with_dup(nums):
    nums.sort(); result = []
    def backtrack(start, path):
        result.append(path[:])
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i-1]: continue
            path.append(nums[i])
            backtrack(i+1, path)
            path.pop()
    backtrack(0, [])
    return result
```

H6. Permutations

```
def permute(nums):
    result = []
    def backtrack(path, used):
        if len(path) == len(nums): result.append(path[:]); return
        for i, n in enumerate(nums):
            if used[i]: continue
            used[i] = True; path.append(n)
            backtrack(path, used)
            path.pop(); used[i] = False
    backtrack([], [False]*len(nums))
    return result
```

H7. Permutations II

```
def permute_unique(nums):
    nums.sort(); result = []
    def backtrack(path, used):
        if len(path) == len(nums): result.append(path[:]); return
        for i in range(len(nums)):
            if used[i]: continue
            if i > 0 and nums[i] == nums[i-1] and not used[i-1]: continue
            used[i] = True; path.append(nums[i])
            backtrack(path, used)
            path.pop(); used[i] = False
    backtrack([], [False]*len(nums))
    return result
```

H8. Combination Sum

```
def combination_sum(candidates, target):
    result = []
    def backtrack(start, path, remaining):
        if remaining == 0: result.append(path[:]); return
        for i in range(start, len(candidates)):
            if candidates[i] > remaining: continue
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i])
            path.pop()
    backtrack(0, [], target)
    return result
```

H9. Word Search

```
def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word): return True
        if r<0 or r>=rows or c<0 or c>=cols or board[r][c] != word[i]: return False
        board[r][c] = "#"
        found = any(dfs(r+dr, c+dc, i+1) for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)])
        board[r][c] = word[i]
        return found
    return any(dfs(r, c, 0) for r in range(rows) for c in range(cols))
```

H10. N-Queens

```
def solve_n_queens(n):
    result = []; cols = set(); diag1 = set(); diag2 = set()
    board = [["."]*n for _ in range(n)]
    def backtrack(r):
        if r == n:
            result.append(["".join(row) for row in board]); return
        for c in range(n):
            if c in cols or r-c in diag1 or r+c in diag2: continue
            cols.add(c); diag1.add(r-c); diag2.add(r+c); board[r][c] = "Q"
            backtrack(r+1)
            cols.remove(c); diag1.remove(r-c); diag2.remove(r+c); board[r][c] = "."
    backtrack(0)
    return result
```

H11. Sudoku Solver

```
def solve_sudoku(board):
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    empties = []
    for r in range(9):
        for c in range(9):
            v = board[r][c]
            if v == '.': empties.append((r, c))
            else:
                rows[r].add(v); cols[c].add(v); boxes[(r//3)*3 + c//3].add(v)
    def backtrack(i):
        if i == len(empties): return True
        r, c = empties[i]; b = (r//3)*3 + c//3
        for d in "123456789":
            if d in rows[r] or d in cols[c] or d in boxes[b]: continue
            rows[r].add(d); cols[c].add(d); boxes[b].add(d); board[r][c] = d
            if backtrack(i+1): return True
            rows[r].remove(d); cols[c].remove(d); boxes[b].remove(d); board[r][c] = '.'
        return False
    backtrack(0)
```

H12. Letter Combinations of Phone Number

```
def letter_combinations(digits):
    if not digits: return []
    mapping = {'2':'abc','3':'def','4':'ghi','5':'jkl','6':'mno','7':'pqrs','8':'tuv','9':'wxyz'}
    result = ['']
    for d in digits:
        result = [prefix + ch for prefix in result for ch in mapping[d]]
    return result
```

Solutions — Category I (Trees)

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val; self.left = left; self.right = right
```

I1. Maximum Depth

```
def max_depth(root):
    if not root: return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

I2. Same Tree

```
def is_same_tree(p, q):
    if not p and not q: return True
    if not p or not q or p.val != q.val: return False
    return is_same_tree(p.left, q.left) and is_same_tree(p.right, q.right)
```

13. Invert Binary Tree

```
def invert_tree(root):
    if not root: return None
    root.left, root.right = invert_tree(root.right), invert_tree(root.left)
    return root
```

14. Symmetric Tree

```
def is_symmetric(root):
    def mirror(a, b):
        if not a and not b: return True
        if not a or not b or a.val != b.val: return False
        return mirror(a.left, b.right) and mirror(a.right, b.left)
    return mirror(root, root)
```

15. Path Sum

```
def has_path_sum(root, target):
    if not root: return False
    if not root.left and not root.right: return target == root.val
    remain = target - root.val
    return has_path_sum(root.left, remain) or has_path_sum(root.right, remain)
```

16. Level Order Traversal

```
from collections import deque
def level_order(root):
    if not root: return []
    result, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft(); level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        result.append(level)
    return result
```

17. Zigzag Level Order

```
from collections import deque
def zigzag_level_order(root):
    if not root: return []
    result, q, ltr = [], deque([root]), True
    while q:
        level = deque()
        for _ in range(len(q)):
            node = q.popleft()
            if ltr: level.append(node.val)
            else: level.appendleft(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        result.append(list(level)); ltr = not ltr
    return result
```

18. Right Side View

```
from collections import deque
def right_side_view(root):
    if not root: return []
    result, q = [], deque([root])
    while q:
        n = len(q)
        for i in range(n):
            node = q.popleft()
            if i == n - 1: result.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
    return result
```

I9. LCA of BST

```
def lowest_common_ancestor_bst(root, p, q):
    while root:
        if p.val < root.val and q.val < root.val: root = root.left
        elif p.val > root.val and q.val > root.val: root = root.right
        else: return root
```

I10. LCA of Binary Tree

```
def lowest_common_ancestor(root, p, q):
    if not root or root is p or root is q: return root
    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)
    if left and right: return root
    return left or right
```

I11. Validate BST

```
def is_valid_bst(root):
    def valid(node, lo, hi):
        if not node: return True
        if not (lo < node.val < hi): return False
        return valid(node.left, lo, node.val) and valid(node.right, node.val, hi)
    return valid(root, float('-inf'), float('inf'))
```

I12. Kth Smallest in BST

```
def kth_smallest(root, k):
    stack = []; curr = root
    while stack or curr:
        while curr:
            stack.append(curr); curr = curr.left
        curr = stack.pop(); k -= 1
        if k == 0: return curr.val
        curr = curr.right
```

I13. Serialize/Deserialize

```
class Codec:
    def serialize(self, root):
        vals = []
        def pre(node):
            if not node: vals.append("#"); return
            vals.append(str(node.val)); pre(node.left); pre(node.right)
        pre(root)
        return ",".join(vals)
    def deserialize(self, data):
        vals = iter(data.split(","))
        def build():
            v = next(vals)
            if v == "#": return None
            node = TreeNode(int(v))
            node.left = build(); node.right = build()
            return node
        return build()
```

I14. Build Tree from Preorder + Inorder

```
def build_tree(preorder, inorder):
    inorder_index = {v: i for i, v in enumerate(inorder)}
    pre_iter = iter(preorder)
    def helper(lo, hi):
        if lo > hi: return None
        val = next(pre_iter); node = TreeNode(val)
        idx = inorder_index[val]
        node.left = helper(lo, idx-1)
        node.right = helper(idx+1, hi)
        return node
    return helper(0, len(inorder)-1)
```

I15. Diameter

```
def diameter(root):
    best = [0]
    def depth(node):
        if not node: return 0
        l, r = depth(node.left), depth(node.right)
        best[0] = max(best[0], l + r)
        return 1 + max(l, r)
    depth(root)
    return best[0]
```

I16. Balanced Binary Tree

```
def is_balanced(root):
    def check(node):
        if not node: return 0
        l = check(node.left)
        if l == -1: return -1
        r = check(node.right)
        if r == -1 or abs(l - r) > 1: return -1
        return 1 + max(l, r)
    return check(root) != -1
```

I17. Max Path Sum

```
def max_path_sum(root):
    best = [float('-inf')]
    def gain(node):
        if not node: return 0
        l = max(0, gain(node.left))
        r = max(0, gain(node.right))
        best[0] = max(best[0], node.val + l + r)
        return node.val + max(l, r)
    gain(root)
    return best[0]
```

I18. Good Nodes

```
def good_nodes(root):
    def dfs(node, mx):
        if not node: return 0
        good = 1 if node.val >= mx else 0
        new_mx = max(mx, node.val)
        return good + dfs(node.left, new_mx) + dfs(node.right, new_mx)
    return dfs(root, float('-inf'))
```

I19. Sum Root to Leaf

```
def sum_numbers(root):
    def dfs(node, total):
        if not node: return 0
        total = total*10 + node.val
        if not node.left and not node.right: return total
        return dfs(node.left, total) + dfs(node.right, total)
    return dfs(root, 0)
```

I20. Flatten to Linked List

```
def flatten(root):
    prev = [None]
    def helper(node):
        if not node: return
        helper(node.right); helper(node.left)
        node.right = prev[0]; node.left = None
        prev[0] = node
    helper(root)
```

Solutions — Category J (Heaps)

J1. Kth Largest Element

```
import heapq
def find_kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]
```

J2. Kth Largest in Stream

```
import heapq
class KthLargest:
    def __init__(self, k, nums):
        self.k = k; self.heap = nums
        heapq.heapify(self.heap)
        while len(self.heap) > k: heapq.heappop(self.heap)
    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k: heapq.heappop(self.heap)
        return self.heap[0]
```

J3. Same as C7.

J4. Median from Stream

```
import heapq
class MedianFinder:
    def __init__(self):
        self.low, self.high = [], [] # low: max-heap (negated), high: min-heap
    def addNum(self, num):
        heapq.heappush(self.low, -num)
        heapq.heappush(self.high, -heapq.heappop(self.low))
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))
    def findMedian(self):
        if len(self.low) > len(self.high): return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2
```

J5. Same as F5.

J6. K Closest Points

```
import heapq
def k_closest(points, k):
    return heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)
```

J7. Task Scheduler

```
from collections import Counter
def least_interval(tasks, n):
    counts = Counter(tasks); mx = max(counts.values())
    max_count = sum(1 for v in counts.values() if v == mx)
    return max(len(tasks), (mx-1)*(n+1) + max_count)
```

J8. Reorganize String

```
import heapq
from collections import Counter
def reorganize(s):
    counts = Counter(s)
    if max(counts.values()) > (len(s) + 1) // 2: return ""
    heap = [(-v, ch) for ch, v in counts.items()]; heapq.heapify(heap)
    prev = (0, ""); result = []
    while heap:
        cnt, ch = heapq.heappop(heap)
        result.append(ch)
        if prev[0] < 0: heapq.heappush(heap, prev)
        prev = (cnt+1, ch)
    return "".join(result)
```

Solutions — Category K (Graphs)

K1. Number of Islands

```
def num_islands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0]); count = 0
    def dfs(r, c):
        if r<0 or r>=rows or c<0 or c>=cols or grid[r][c] != '1': return
        grid[r][c] = '0'
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            dfs(r+dr, c+dc)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1; dfs(r, c)
    return count
```

K2. Max Area of Island

```
def max_area_of_island(grid):
    rows, cols = len(grid), len(grid[0])
    def dfs(r, c):
        if r<0 or r>=rows or c<0 or c>=cols or grid[r][c] != 1: return 0
        grid[r][c] = 0
        return 1 + dfs(r+1,c) + dfs(r-1,c) + dfs(r,c+1) + dfs(r,c-1)
    return max((dfs(r,c) for r in range(rows) for c in range(cols)), default=0)
```

K3. Flood Fill

```
def flood_fill(image, sr, sc, color):
    original = image[sr][sc]
    if original == color: return image
    rows, cols = len(image), len(image[0])
    def dfs(r, c):
        if r<0 or r>=rows or c<0 or c>=cols or image[r][c] != original: return
        image[r][c] = color
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]: dfs(r+dr, c+dc)
    dfs(sr, sc)
    return image
```

K4. Rotting Oranges

```
from collections import deque
def oranges_rotting(grid):
    q = deque(); fresh = 0
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 2: q.append((r, c, 0))
            elif v == 1: fresh += 1
    minutes = 0
    while q:
        r, c, t = q.popleft(); minutes = t
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
            if 0<=nr<len(grid) and 0<=nc<len(grid[0]) and grid[nr][nc] == 1:
                grid[nr][nc] = 2; fresh -= 1
                q.append((nr, nc, t+1))
    return -1 if fresh else minutes
```

K5. Walls and Gates

```
from collections import deque
def walls_and_gates(rooms):
    q = deque()
    for r in range(len(rooms)):
        for c in range(len(rooms[0])):
            if rooms[r][c] == 0: q.append((r, c))
    while q:
        r, c = q.popleft()
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
            if 0<=nr<len(rooms) and 0<=nc<len(rooms[0]) and rooms[nr][nc] > rooms[r][c] + 1:
                rooms[nr][nc] = rooms[r][c] + 1
                q.append((nr, nc))
```

K6. Course Schedule

```

from collections import defaultdict, deque
def can_finish(num_courses, prerequisites):
    graph = defaultdict(list); indeg = [0]*num_courses
    for a, b in prerequisites:
        graph[b].append(a); indeg[a] += 1
    q = deque(i for i in range(num_courses) if indeg[i] == 0)
    done = 0
    while q:
        node = q.popleft(); done += 1
        for nxt in graph[node]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0: q.append(nxt)
    return done == num_courses

```

K7. Course Schedule II

```

from collections import defaultdict, deque
def find_order(num_courses, prerequisites):
    graph = defaultdict(list); indeg = [0]*num_courses
    for a, b in prerequisites:
        graph[b].append(a); indeg[a] += 1
    q = deque(i for i in range(num_courses) if indeg[i] == 0)
    order = []
    while q:
        node = q.popleft(); order.append(node)
        for nxt in graph[node]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0: q.append(nxt)
    return order if len(order) == num_courses else []

```

K8. Clone Graph

```

def clone_graph(node):
    if not node: return None
    old_to_new = {}
    def dfs(n):
        if n in old_to_new: return old_to_new[n]
        clone = Node(n.val)
        old_to_new[n] = clone
        clone.neighbors = [dfs(nb) for nb in n.neighbors]
        return clone
    return dfs(node)

```

K9. Pacific Atlantic Water Flow

```

def pacific_atlantic(heights):
    if not heights: return []
    rows, cols = len(heights), len(heights[0])
    pac, atl = set(), set()
    def dfs(r, c, visited):
        visited.add((r, c))
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited and heights[nr][nc] >= heights[r][c]:
                dfs(nr, nc, visited)
    for r in range(rows):
        dfs(r, 0, pac); dfs(r, cols-1, atl)
    for c in range(cols):
        dfs(0, c, pac); dfs(rows-1, c, atl)
    return list(pac & atl)

```

K10. Word Ladder

```

from collections import defaultdict, deque
def ladder_length(begin, end, word_list):
    if end not in word_list: return 0
    L = len(begin); patterns = defaultdict(list)
    for w in word_list:
        for i in range(L):
            patterns[w[:i] + "*" + w[i+1:]].append(w)
    visited = {begin}; q = deque([(begin, 1)])
    while q:
        word, steps = q.popleft()
        for i in range(L):
            key = word[:i] + "*" + word[i+1:]
            for w in patterns[key]:
                if w == end: return steps + 1
                if w not in visited:
                    visited.add(w); q.append((w, steps + 1))
    return 0

```

K11. Connected Components

```
def count_components(n, edges):
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]; x = parent[x]
        return x
    count = n
    for a, b in edges:
        pa, pb = find(a), find(b)
        if pa != pb: parent[pa] = pb; count -= 1
    return count
```

K12. Graph Valid Tree

```
def valid_tree(n, edges):
    if len(edges) != n - 1: return False
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]; x = parent[x]
        return x
    for a, b in edges:
        pa, pb = find(a), find(b)
        if pa == pb: return False
        parent[pa] = pb
    return True
```

K13. Dijkstra

```
import heapq
def dijkstra(graph, source):
    dist = {node: float('inf') for node in graph}
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]: continue
        for nb, w in graph[node]:
            nd = d + w
            if nd < dist[nb]:
                dist[nb] = nd; heapq.heappush(heap, (nd, nb))
    return dist
```

K14. Network Delay Time

```
import heapq
from collections import defaultdict
def network_delay_time(times, n, k):
    graph = defaultdict(list)
    for u, v, w in times: graph[u].append((v, w))
    dist = {i: float('inf') for i in range(1, n+1)}
    dist[k] = 0; heap = [(0, k)]
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]: continue
        for nb, w in graph[node]:
            if d + w < dist[nb]:
                dist[nb] = d + w; heapq.heappush(heap, (dist[nb], nb))
    return -1 if float('inf') in dist.values() else max(dist.values())
```

K15. Alien Dictionary

```

from collections import defaultdict, deque
def alien_order(words):
    graph = defaultdict(set); indeg = {c: 0 for w in words for c in w}
    for a, b in zip(words, words[1:]):
        for x, y in zip(a, b):
            if x != y:
                if y not in graph[x]:
                    graph[x].add(y); indeg[y] += 1
                break
            else:
                if len(a) > len(b): return ""
    q = deque([c for c, d in indeg.items() if d == 0]); result = []
    while q:
        c = q.popleft(); result.append(c)
        for nxt in graph[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0: q.append(nxt)
    return "".join(result) if len(result) == len(indeg) else ""

```

Solutions — Category L (Dynamic Programming)

L1. Same as H3.

L2. House Robber

```

def rob(nums):
    prev = curr = 0
    for n in nums:
        prev, curr = curr, max(curr, prev + n)
    return curr

```

L3. House Robber II

```

def rob2(nums):
    if len(nums) == 1: return nums[0]
    def rob_line(arr):
        prev = curr = 0
        for n in arr:
            prev, curr = curr, max(curr, prev + n)
        return curr
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))

```

L4. Coin Change

```

def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1); dp[0] = 0
    for i in range(1, amount + 1):
        for c in coins:
            if c <= i: dp[i] = min(dp[i], dp[i-c] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

```

L5. Coin Change II

```

def change(amount, coins):
    dp = [0] * (amount + 1); dp[0] = 1
    for c in coins:
        for i in range(c, amount + 1):
            dp[i] += dp[i - c]
    return dp[amount]

```

L6. Longest Increasing Subsequence

```

import bisect
def length_of_lis(nums):
    piles = []
    for n in nums:
        i = bisect.bisect_left(piles, n)
        if i == len(piles): piles.append(n)
        else: piles[i] = n
    return len(piles)

```

L7. Longest Common Subsequence

```
def lcs(a, b):
    dp = [[0]*(len(b)+1) for _ in range(len(a)+1)]
    for i in range(1, len(a)+1):
        for j in range(1, len(b)+1):
            if a[i-1] == b[j-1]: dp[i][j] = dp[i-1][j-1] + 1
            else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[-1][-1]
```

L8. Edit Distance

```
def min_distance(a, b):
    m, n = len(a), len(b)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1): dp[i][0] = i
    for j in range(n+1): dp[0][j] = j
    for i in range(1, m+1):
        for j in range(1, n+1):
            if a[i-1] == b[j-1]: dp[i][j] = dp[i-1][j-1]
            else: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]
```

L9. Unique Paths

```
def unique_paths(m, n):
    dp = [1]*n
    for _ in range(1, m):
        for j in range(1, n): dp[j] += dp[j-1]
    return dp[-1]
```

L10. Minimum Path Sum

```
def min_path_sum(grid):
    m, n = len(grid), len(grid[0])
    for i in range(m):
        for j in range(n):
            if i == 0 and j == 0: continue
            if i == 0: grid[i][j] += grid[i][j-1]
            elif j == 0: grid[i][j] += grid[i-1][j]
            else: grid[i][j] += min(grid[i-1][j], grid[i][j-1])
    return grid[-1][-1]
```

L11. Same as B15.

L12. Partition Equal Subset Sum

```
def can_partition(nums):
    total = sum(nums)
    if total % 2: return False
    target = total // 2
    dp = {0}
    for n in nums:
        dp = dp | {x + n for x in dp}
        if target in dp: return True
    return False
```

L13. Stock with Cooldown

```
def max_profit_cooldown(prices):
    hold, sold, rest = float('-inf'), 0, 0
    for p in prices:
        prev_sold = sold
        sold = hold + p
        hold = max(hold, rest - p)
        rest = max(rest, prev_sold)
    return max(sold, rest)
```

L14. Decode Ways

```
def num_decodings(s):
    if not s or s[0] == '0': return 0
    prev, curr = 1, 1
    for i in range(1, len(s)):
        temp = 0
        if s[i] != '0': temp += curr
        if 10 <= int(s[i-1:i+1]) <= 26: temp += prev
        prev, curr = curr, temp
    return curr
```

L15. Longest Palindromic Subsequence

```
def longest_palindrome_subseq(s):
    n = len(s); dp = [[0]*n for _ in range(n)]
    for i in range(n): dp[i][i] = 1
    for length in range(2, n+1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j]:
                dp[i][j] = dp[i+1][j-1] + 2
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])
    return dp[0][n-1]
```

Solutions — Category M (Greedy, Bits, Design)

M1. Jump Game

```
def can_jump(nums):
    farthest = 0
    for i, n in enumerate(nums):
        if i > farthest: return False
        farthest = max(farthest, i + n)
    return True
```

M2. Jump Game II

```
def jump(nums):
    jumps = end = farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == end:
            jumps += 1; end = farthest
    return jumps
```

M3. Gas Station

```
def can_complete_circuit(gas, cost):
    if sum(gas) < sum(cost): return -1
    tank = start = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0: start = i + 1; tank = 0
    return start
```

M4. Partition Labels

```
def partition_labels(s):
    last = {c: i for i, c in enumerate(s)}
    result = []; start = end = 0
    for i, c in enumerate(s):
        end = max(end, last[c])
        if i == end:
            result.append(end - start + 1); start = i + 1
    return result
```

M5. Meeting Rooms II

```
import heapq
def min_meeting_rooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []
    for start, end in intervals:
        if heap and heap[0] <= start: heapq.heappop(heap)
        heapq.heappush(heap, end)
    return len(heap)
```

M6. Single Number

```
def single_number(nums):
    result = 0
    for n in nums: result ^= n
    return result
```

M7. Number of 1 Bits

```
def hamming_weight(n):
    count = 0
    while n:
        n &= n - 1; count += 1
    return count
```

M8. Counting Bits

```
def count_bits(n):
    dp = [0]*(n+1)
    for i in range(1, n+1):
        dp[i] = dp[i >> 1] + (i & 1)
    return dp
```

M9. Missing Number

```
def missing_number(nums):
    result = len(nums)
    for i, n in enumerate(nums):
        result ^= i ^ n
    return result
```

M10. Reverse Bits

```
def reverse_bits(n):
    result = 0
    for _ in range(32):
        result = (result << 1) | (n & 1); n >>= 1
    return result
```

M11. Sum of Two Integers

```
def get_sum(a, b):
    MASK = 0xFFFFFFFF; MAX_INT = 0x7FFFFFFF
    while b:
        a, b = (a ^ b) & MASK, ((a & b) << 1) & MASK
    return a if a <= MAX_INT else ~(a ^ MASK)
```

M12. Design Twitter

```

import heapq
from collections import defaultdict
class Twitter:
    def __init__(self):
        self.time = 0
        self.tweets = defaultdict(list) # user -> [(time, tweetId)]
        self.follows = defaultdict(set)
    def postTweet(self, user, tid):
        self.tweets[user].append((self.time, tid)); self.time += 1
    def getNewsFeed(self, user):
        users = self.follows[user] | {user}
        candidates = [t for u in users for t in self.tweets[u]]
        return [tid for _, tid in heapq.nlargest(10, candidates)]
    def follow(self, follower, followee):
        self.follows[follower].add(followee)
    def unfollow(self, follower, followee):
        self.follows[follower].discard(followee)

```

M13. Design Tic-Tac-Toe

```

class TicTacToe:
    def __init__(self, n):
        self.n = n
        self.rows = [0]*n; self.cols = [0]*n
        self.diag = 0; self.anti = 0
    def move(self, r, c, player):
        val = 1 if player == 1 else -1
        self.rows[r] += val; self.cols[c] += val
        if r == c: self.diag += val
        if r + c == self.n - 1: self.anti += val
        if abs(self.rows[r]) == self.n or abs(self.cols[c]) == self.n \
            or abs(self.diag) == self.n or abs(self.anti) == self.n:
            return player
        return 0

```

M14. Design Snake Game

```

from collections import deque
class SnakeGame:
    def __init__(self, width, height, food):
        self.w, self.h = width, height
        self.food = deque(food); self.snake = deque([(0,0)])
        self.body = {(0,0)}; self.score = 0
    def move(self, direction):
        head = self.snake[0]
        dr, dc = {'U':(-1,0), 'D':(1,0), 'L':(0,-1), 'R':(0,1)}[direction]
        nr, nc = head[0]+dr, head[1]+dc
        if nr<0 or nr>=self.h or nc<0 or nc>=self.w: return -1
        if self.food and [nr, nc] == self.food[0]:
            self.food.popleft(); self.score += 1
        else:
            tail = self.snake.pop(); self.body.discard(tail)
        if (nr, nc) in self.body: return -1
        self.snake.appendleft((nr, nc)); self.body.add((nr, nc))
        return self.score

```

APPENDIX — YOU'RE READY: WHAT'S NEXT

After finishing this document you have covered ~180 problems across every pattern that appears in interviews. If you want to keep going:

1. **NeetCode 150 / Blind 75** — you'll recognize almost every problem. Speed-solve them for reflex-building.
2. **LeetCode weekly contests** — best simulation of interview pressure. Aim to solve 2/4 problems in the first month; 3/4 after that.
3. **Mock interviews** — Pramp, interviewing.io, or a peer. The "talking while coding" skill only forms through practice under observation.
4. **System design deep-dives** — Alex Xu's "System Design Interview" volumes 1 and 2 are the gold standard for senior interviews.

One final piece of advice

You mentioned hating LeetCode-style tests. That feeling doesn't fully disappear — even engineers who love this work still sometimes groan when they open a fresh problem. What changes is your *relationship* with the discomfort: from "I don't know if I can" to "I know I've solved 150 of these, this one is just another." Confidence is the accumulation of small wins.

Two problems a day. Every day. In two months you will be a completely different candidate. Not because you got smarter — but because you built the reps.

Good luck. You've got this.